



COMPLETE TECHNICAL REPORT · BUILD HISTORY, ARCHITECTURE AND CODE WALKTHROUGH

Real-Time Pseudo-Streaming Credit Card Fraud Detection

Every card transaction is scored on arrival in under a millisecond, mapped onto calibrated risk bands, aggregated into account-level risk, explained with native TreeSHAP contributions, and worked through an analyst investigation workbench. This document explains what we built, why each decision was made, and what every module in the codebase does.

0.97 ms AVG INFERENCE	0.76 TEST PR-AUC	0.03 % FALSE POSITIVE RATE	60 / 60 TESTS PASSING
---	--	--	---

Contents

This report is written to be read at three depths. Sections 1 to 6 give the story and the architecture, which is enough for a review conversation. Sections 7 to 15 are the engineering deep dive, module by module. Sections 16 to 24 cover quality, operations and the questions a panel is most likely to ask.

1	Executive summary	What was built and what it measurably achieves
2	Problem statement and objectives	Why batch fraud detection fails and what we targeted
3	Build journey: initial state to now	Eight phases, in the order they were actually built
4	Languages, frameworks and libraries	Every dependency, its version and why it is there
5	System architecture	Three tiers, three threads, one contract
6	End-to-end data flow	The life of a single transaction, step by step
7	The dataset and exploratory analysis	283,726 transactions, 473 frauds, 1:599 imbalance
8	Machine learning pipeline	Preprocessing, features, four candidates, threshold tuning
9	Risk engine mathematics	Probability to risk score, bands, seven account signals
10	Backend code walkthrough	Every Python module explained
11	API reference	All 21 endpoints with validation and errors
12	Database layer	Seven tables, RLS, column grants, Realtime
13	Frontend architecture	Component tree, routing, state, styling system
14	Frontend code walkthrough	Every page and component explained
15	Advanced feature: investigation workbench	Explainable AI plus analyst case management
16	Security model	Key custody, RLS, validation, known trade-offs
17	Performance engineering	Where the 50 ms budget actually goes
18	Testing and verification	60 unit tests, smoke harness, browser flows, CI
19	Engineering rigour: a bug we found and fixed	Stale investigation cases after a stream reset
20	Running the project	Exact commands for engine, database and dashboard
21	Concept glossary	Every technical term used, in plain language
22	Limitations and roadmap	Honest boundaries and the next increments
23	Panel question preparation	Likely questions with evidence-backed answers

How to read the numbers in this report

Every metric quoted here is taken from artifacts produced by the project itself, not from estimates. Model numbers come from `ml-engine/models/model_metadata.json`, written by the training run. Dataset numbers come from the same file's embedded profile. Latency numbers come from the benchmark that runs through the real serving code path. Test counts come from the pytest run. Where a figure could not be verified, this report says so explicitly.

1 Executive summary

FraudStream AI is a complete fraud detection system, built from an empty folder to a running three-tier application. It ingests historical card transaction data, trains and compares four detection models, replays unseen transactions as a live event stream, scores each one in under a millisecond, escalates risky accounts, explains individual decisions, and lets a human analyst work each case to a documented outcome.

What exists today

DELIVERED CAPABILITY BY LAYER

Layer	What it does	Evidence it works
ML pipeline	Cleans the dataset, splits it chronologically, engineers five time and amount features, trains four candidate models, tunes the decision threshold, and serialises the winner.	Reproducible run recorded in <code>model_metadata.json</code> with per-candidate metrics and a leakage check.
Streaming engine	A Python generator emits one transaction per iteration; a background thread scores it, updates account risk, raises alerts and hands off persistence.	A test asserts that only three records are parsed when three events are consumed, proving no batch read.
Risk engine	Rescales model probability onto four action-oriented bands and aggregates seven weighted behavioural signals per account.	17 risk tests, including a monotonicity sweep and a weight-normalisation assertion.
API	21 FastAPI endpoints for prediction, stream control, dashboard data, investigations, accounts, model and dataset operations.	Live smoke run and 17 API test cases, including six path-traversal rejection cases.
Database	Seven Postgres tables with row level security, column-level grants and Realtime publication.	Two idempotent SQL migrations that can be re-run safely.
Dashboard	A public landing site plus a seven-page authenticated console with live feed, charts, triage and the investigation workbench.	Production build succeeds; browser harness drives the real user flows with zero console errors.
Explainability	Native XGBoost TreeSHAP contributions plus behavioural reason codes attached to every alert, surfaced in a case drawer.	Verified end to end: eight ranked feature contributions on a live critical case.

Headline results

0.9739 ms AVERAGE INFERENCE p95 1.588 ms, p99 1.932 ms against a 50 ms budget	0.7629 TEST PR-AUC The honest metric at 1:599 imbalance	0.9737 TEST ROC-AUC Ranking quality across all thresholds	0.000306 FALSE POSITIVE RATE 13 false alarms in 42,508 legitimate rows
---	---	---	--

The single most important number

p95 inference latency is **1.588 ms against a 50 ms requirement**, which is roughly 31 times inside budget. That headroom is not an accident: latency is a hard gate in model selection, and the winning model was chosen partly because a more accurate competitor was too slow. Section 8 shows exactly how that trade-off was made and section 17 shows where the milliseconds go.

What makes this more than a classifier notebook

- **It streams.** The processing loop handles one transaction at a time through a real Python generator, so the architecture matches how payment authorisation actually works rather than scoring a CSV in bulk.
- **It is honest about imbalance.** Accuracy is never reported anywhere in the codebase. With 0.1667 % fraud, a model that predicts "legitimate" every time scores 99.83 % accuracy and catches nothing.
- **It tunes the decision, not just the model.** The threshold is 0.184152, chosen by maximising F2 subject to a precision floor of 0.50, rather than being left at the default 0.5.
- **It explains itself.** Every alert carries additive TreeSHAP contributions in log-odds space plus human-readable reason codes, so an analyst sees why the model fired.
- **It closes the loop.** Analysts assign, annotate, transition and resolve cases with optimistic concurrency control, and the resolution codes become measurable feedback.
- **It is verified.** 60 automated tests, an end-to-end smoke harness with 23 assertions, and a browser harness that drives real interactions.

2 Problem statement and objectives

2.1 The business problem

Card fraud has to be stopped while the transaction is still in flight. Once an authorisation is approved, the money has moved and the institution is into chargeback and recovery, which is slower and more expensive than prevention. That creates three constraints that pull against each other:

Constraint	Why it is hard	How this project answers it
Speed	A scoring decision sits inside the payment authorisation path. Tens of milliseconds of extra latency degrades checkout conversion.	Hard latency requirement of 50 ms, measured through the real serving path, enforced as a gate during model selection. Achieved p95 of 1.588 ms.
Rarity	Fraud is 0.1667 % of transactions. Standard accuracy and default thresholds are actively misleading at that ratio.	PR-AUC as the selection metric, class reweighting inside every model, and a tuned threshold with an explicit precision floor.
Trust	A score alone is not actionable. An analyst must know why a transaction was flagged, and a blocked customer deserves a reason.	TreeSHAP feature contributions plus behavioural reason codes on every alert, presented in an investigation workbench with an audit trail.

2.2 Why "pseudo-streaming"

A production system would consume a message broker such as Kafka. We do not have a live card payment feed, and inventing one would prove nothing. Instead we take the final 15 % of a real dataset's timeline, which the model has never seen, and replay it in original chronological order through a Python generator, one transaction per iteration, with a configurable inter-arrival delay.

Why this is the right simplification

The property that matters for the architecture is **one-record-at-a-time processing with bounded memory and a clean stop**. A generator gives exactly that. Swapping the generator for a Kafka consumer changes the source of the loop, not the loop itself, because everything downstream already assumes single-event handling. The dataset order is never shuffled, so no future transaction can influence a past decision.

2.3 Objectives we set and met

ID	Objective	Target	Achieved	Status
PR-001	Per-transaction prediction latency	< 50 ms	0.97 ms avg, 1.59 ms p95	Met
PR-002	Process transactions one at a time	Generator based	Generator, test-enforced	Met
PR-003	Dashboard updates near real time	Live	2 s poll plus Realtime push	Met
PR-004	Reliability under malformed input	Never crash the stream	Invalid rows counted and skipped	Met
FR-006	Model selection on quality and latency	Both criteria	Latency gate plus best PR-AUC	Met

ID	Objective	Target	Achieved	Status
FR-007	Threshold tuning	Not 0.5	0.184152 by F2 with precision floor	Met
FR-010	Fraud alerting	Automatic	Raised at risk score 0.70	Met
FR-011	High-risk account detection	Aggregated	Seven weighted signals per account	Met
FR-014	Authentication	Required	Supabase JWT verified per request	Met with caveat
PR-005	Scalable to a real broker	Design for it	Interface ready, not implemented	Designed only

The FR-014 caveat and the PR-005 boundary are explained honestly in sections 16 and 22. We prefer to state them here rather than let a reviewer discover them.

3 Build journey: initial state to now

The project began with a requirements document (`docs/PRD.json`) and a raw CSV file. Nothing else. The build order below is the order the code was actually written, and each phase depended on the one before it being verifiable.

3.1 The eight phases

PHASE BY PHASE, WITH THE ARTIFACT EACH ONE PRODUCED

#	Phase	What we did and why	Artifact produced
0	Initial state	A requirements specification with 14 functional requirements, 5 performance requirements and 10 success criteria, plus <code>creditcard.csv</code> sitting outside the repository because it is roughly 150 MB.	<code>docs/PRD.json</code>
1	Project setup	Repository skeleton, pinned dependency lists, environment variable templates, and one central configuration module so that training, streaming and the API could never disagree about paths, feature order, risk bands or thresholds.	<code>ml-engine/src/config.py</code> , <code>requirements.txt</code> , <code>.env.example</code>
2	Dataset and EDA	Schema validation, a four-step cleaning report, class distribution, hour-of-day analysis and label correlations. We deliberately put the analysis inside the pipeline instead of a notebook so it regenerates on every training run and cannot drift.	<code>data/dataset_profile.json</code>
3	ML pipeline	Chronological split, feature engineering with a vectorised and a scalar path, imbalance handling, four candidate models, imbalance-aware evaluation, threshold tuning, a latency benchmark through the serving path, and latency-gated selection.	<code>fraud_model.joblib</code> , <code>preprocessor.joblib</code> , <code>model_metadata.json</code>
4	Streaming engine	The generator, the single-transaction processing loop on a background thread, thread-safe rolling state with ring buffers, the risk engine, account aggregation, and the fraud index that lets a demo start near labelled fraud.	<code>data/stream_test.csv</code> , <code>stream_fraud_index.json</code>
5	Supabase integration	Schema migration with row level security, a buffered background writer so network latency never touches the inference budget, an auth trigger for user profiles, and Realtime publication for live push.	<code>supabase/migrations/0001_init.sql</code>
6	React dashboard	A public landing site, an authenticated console with a shared live-data context, five data pages, charts, filters, alert triage, and an accessible design system built on Tailwind primitives.	<code>frontend/dist</code> production bundle

#	Phase	What we did and why	Artifact produced
7	Testing	60 unit and integration tests, an end-to-end smoke script that spawns a real server and asserts 23 checks, and Playwright harnesses that drive genuine user interactions and screenshot every page.	<code>tests/</code> , <code>scripts/smoke_e2e.py</code> , <code>tools/*.cjs</code>
8	Advanced feature	The explainable investigation workbench: TreeSHAP contributions, behavioural reason codes, a thread-safe case store with optimistic concurrency, seven new API endpoints, two new tables, and a full React case-management page.	<code>investigations/store.py</code> , <code>0002_investigations.sql</code> , <code>pages/Investigations.jsx</code>

3.2 Decisions that shaped everything after them

Decision	Alternative we rejected	Consequence
One central config module	Constants defined near where they are used	Feature order, risk bands and the latency target exist in exactly one place, so the training path and the serving path cannot silently diverge.
Chronological split	Random <code>train_test_split</code>	No future information leaks backwards. It also makes the test split usable as a realistic replay stream, which the whole streaming demo depends on.
Generator, not DataFrame	<code>pandas.read_csv</code> then iterate rows	Bounded memory, honest per-transaction latency, and a clean mid-file stop. A test proves laziness by counting parse calls.
Latency as a selection gate	Pick the highest PR-AUC model	Random Forest scored higher (0.8715 vs 0.8472) but needed 33.16 ms at p95. XGBoost ships instead at 1.76 ms, keeping 96 % of the budget free.
Two feature code paths	Reuse the vectorised path for single records	Allocating a DataFrame per transaction would waste most of the latency budget. A parity test keeps the scalar and vectorised paths numerically identical.
Background persistence writer	Write to the database inline	Network round trips never count against the 50 ms budget. The queue sheds load rather than blocking the stream.
Derived identity, never a feature	Feed account and merchant into the model	The dataset is anonymised, so account identity is a deterministic hash. Using it as a feature would be circular. A test asserts none of these keys appear in the model feature list.
In-memory rolling state	Query Postgres for every dashboard refresh	The dashboard works with no database configured, and <code>/api/metrics</code> never runs an aggregate query on the hot path.

The through-line

Every one of those decisions trades a small amount of convenience for a property that can be demonstrated: bounded memory, measurable latency, no leakage, or no drift between two code paths. That is why the test suite is able to assert architectural properties and not just output values.

4 Languages, frameworks and libraries

Two programming languages, one query language and one styling language. Every dependency is pinned to an exact version, with no floating ranges, so a rebuild months later produces the same behaviour.

4.1 Languages used

Language	Version	Where it is used	Why this language
Python	3.10+ (CI on 3.12, local venv 3.12.13)	ML pipeline, streaming engine, risk engine, REST API, tests	The mature numerical and machine learning ecosystem lives here. scikit-learn, XGBoost, pandas and NumPy are all first-class, and FastAPI gives production-grade HTTP without leaving the language.
JavaScript (JSX)	ES modules, React 16 classic JSX transform	Landing site, authenticated console, all 10 pages and 12 component modules	React is the specified frontend framework. JSX keeps markup and the logic that drives it in one place, which suits a dashboard where nearly every element is data-bound.
SQL (PostgreSQL)	Postgres via Supabase, pgcrypto extension	Schema, constraints, indexes, triggers, row level security policies, Realtime publication	Security is enforced in the database, not just in application code. Row level security and column-level grants mean a stolen browser token still cannot rewrite a risk score.
PowerShell	Windows PowerShell 5.1	15 automation scripts for training, serving, testing and verification	The development machine is Windows. These wrappers make every long command a single repeatable invocation with logs written to <code>logs/</code> .
CSS (Tailwind)	Tailwind 3.4.17 with a custom theme	Entire visual system, plus 21 hand-written component classes	Utility classes keep styling next to markup while the custom theme enforces one palette, one type scale and one shadow language across 22 component files.

4.2 Backend dependencies, pinned

Package	Version	Role	What we actually use it for
<code>fastapi</code>	0.115.6	API framework	21 routes, dependency injection for auth, automatic OpenAPI docs at <code>/docs</code> , and typed request validation through Pydantic.
<code>uvicorn[standard]</code>	0.34.0	ASGI server	Runs the application. The <code>standard</code> extra brings efficient event loop and HTTP parsing.
<code>pydantic</code>	2.10.5	Validation	Seven request schemas. Enforces numeric ranges, regex-constrained status values, string length caps and the path-traversal guard on stream source names.
<code>python-multipart</code>	0.0.20	Uploads	Parses <code>multipart/form-data</code> for the dataset CSV upload endpoint.
<code>httpx</code>	0.28.1	HTTP client	Async verification of Supabase access tokens, and the synchronous client inside the smoke test harness.
<code>pandas</code>	2.2.3	Data frames	Offline only: loading, cleaning, chronological sorting, splitting and the EDA profile. Deliberately absent from the streaming hot path.
<code>numpy</code>	2.1.3	Arrays and math	The single-record feature vector, the numerically stable sigmoid, percentile maths and TreeSHAP contribution handling.

Package	Version	Role	What we actually use it for
scikit-learn	1.6.1	ML toolkit	Pipeline, SimpleImputer, StandardScaler, LogisticRegression, RandomForestClassifier, IsolationForest, HistGradientBoosting fallback, and every metric including <code>average_precision_score</code> .
xgboost	2.1.3	Selected model	The production classifier, and the native <code>pred_contribs=True</code> TreeSHAP call that powers explainability.
joblib	1.4.2	Serialisation	Persists the fitted model and the fitted preprocessor so the API loads them once at startup.
imbalanced-learn	0.13.0	Resampling	Optional SMOTE oversampling behind a <code>--smote</code> flag. Off by default; class weighting is the primary strategy.
supabase	2.11.0	Database client	Batched inserts and upserts from the background writer thread, and the synchronous alert status mirror.
python-dotenv	1.0.1	Config	Loads <code>ml-engine/.env</code> . Imported defensively so tests run even if it is absent.
pytest	8.3.4	Testing	60 test cases with fixtures, parametrisation and automatic skipping when trained artifacts are unavailable.

4.3 Frontend dependencies, pinned

Package	Version	What we use it for
react / react-dom	16.14.0	The UI library. React 16 means <code>ReactDOM.render</code> rather than <code>createRoot</code> , and the classic JSX transform, so every component file imports React explicitly.
react-router-dom	5.3.4	Routing. Version 5 is deliberate because the sidebar uses <code>NavLink activeClassName</code> and <code>exact</code> , which do not exist in version 6.
axios	1.7.9	HTTP client with a request interceptor that attaches the Supabase bearer token to every call, a 20 s default timeout and a 300 s timeout for uploads.
@supabase/supabase-js	2.48.1	Browser auth (sign in, sign up, reset, session refresh) and the Realtime websocket subscription, throttled to 20 events per second.
recharts	2.12.7	Four charts: throughput area chart, latency line chart, precision-recall curve and hourly distribution bar chart.
vite	5.4.11	Dev server and production bundler. Configured with file-watch polling because the project lives inside a OneDrive-synced folder where native file events are unreliable.
tailwindcss	3.4.17	Styling with a custom theme: two colour families, one extra type size, one letter-spacing token, three shadows and two animations.
@vitejs/plugin-react	4.3.4	React integration for Vite, configured with <code>jsxRuntime: 'classic'</code> for React 16 compatibility.
postcss / autoprefixer	8.4.49 / 10.4.20	Tailwind runs as a PostCSS plugin; Autoprefixer targets the declared browserslist.

4.4 Models trained and compared

FOUR CANDIDATES, ALL TRAINED ON IDENTICAL DATA WITH IDENTICAL PREPROCESSING

Model	Type	Hyperparameters actually used	Imbalance strategy
-------	------	-------------------------------	--------------------

Model	Type	Hyperparameters actually used	Imbalance strategy
XGBoost selected	Gradient-boosted trees	<code>n_estimators=400, max_depth=5, learning_rate=0.08, subsample=0.9, colsample_bytree=0.9, reg_lambda=1.0, tree_method="hist", eval_metric="aucpr"</code>	<code>scale_pos_weight = negatives / positives</code> , so roughly 599
Random Forest	Bagged trees	<code>n_estimators=160, min_samples_leaf=3, max_features="sqrt", n_jobs=-1</code>	<code>class_weight="balanced_subsample"</code>
Logistic Regression	Linear baseline	<code>max_iter=1000, solver="lbfgs"</code>	<code>class_weight="balanced"</code>
Isolation Forest	Unsupervised anomaly detection	<code>n_estimators=100, max_samples=4096, contamination=observed_fraud_rate</code>	Contamination parameter; trained on features only, no labels
HistGradientBoosting	Fallback only	<code>max_iter=400, learning_rate=0.1</code>	<code>class_weight="balanced"</code> . Used only if the XGBoost import fails, so the pipeline never dies from a missing optional dependency.

Why an unsupervised model was included at all

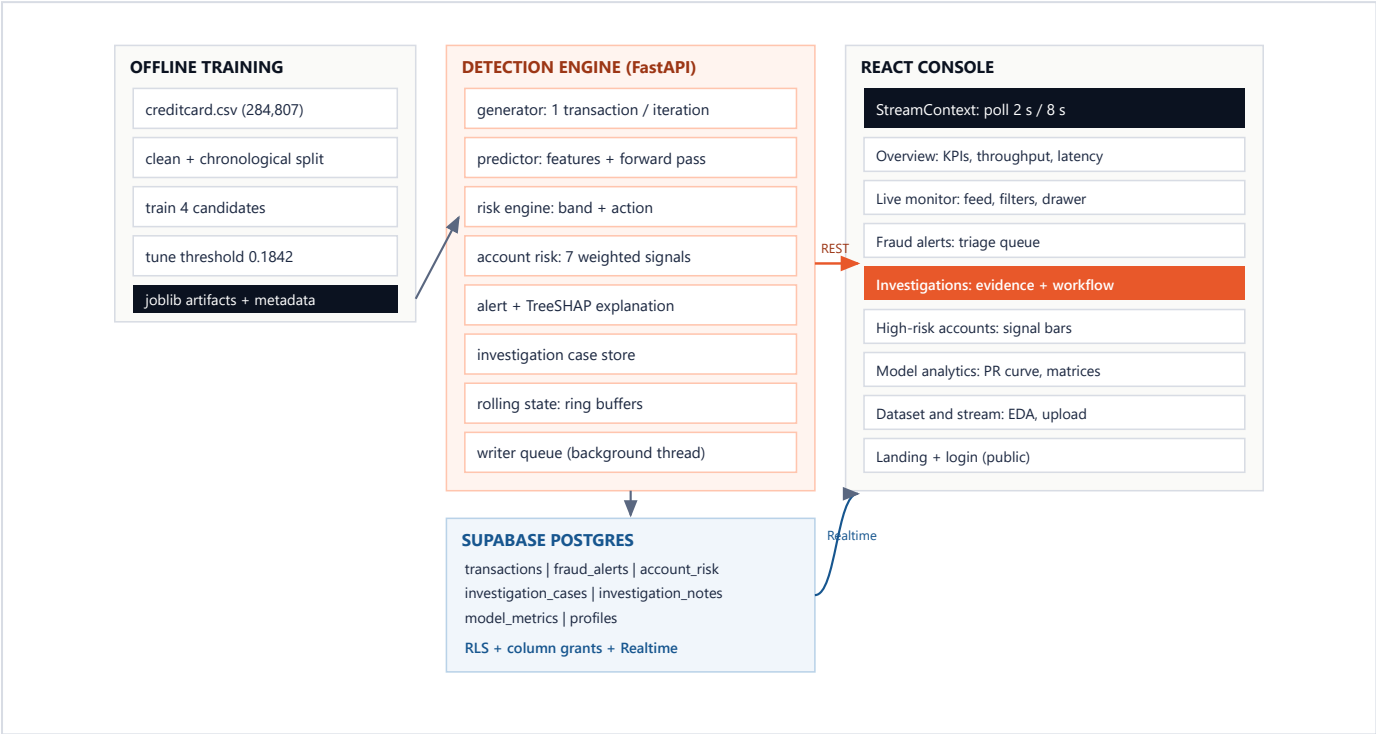
Isolation Forest scored a PR-AUC of 0.0553 against XGBoost's 0.8472, so it lost decisively. That is the point of including it. Real fraud systems often start with anomaly detection because labels are scarce, and having the number in the report lets us state with evidence that supervised learning on labelled history is roughly fifteen times better here, rather than merely asserting it.

4.5 Infrastructure and tooling

Component	Technology	Purpose
Database	Supabase Postgres	Seven tables, row level security, column grants, triggers, indexes
Authentication	Supabase Auth	Email and password sessions, JWT access tokens verified by the API
Live push	Supabase Realtime	Postgres logical replication pushed to the browser over a websocket
Continuous integration	GitHub Actions	Two jobs: pytest on Python 3.12, and a production frontend build on Node 20
Browser automation	Playwright with Microsoft Edge	Four harnesses: interaction flows, console screenshots, landing screenshots, 404 detection
Target hosting	Vercel, Render, Supabase	Documented in <code>docs/deployment.md</code> , including the single-worker constraint

5 System architecture

5.1 The three tiers



5.2 The three threads inside the engine

Concurrency here is deliberate and small. Three kinds of thread exist, each with one job, and the boundaries between them are the reason the latency budget holds.

Thread	Created by	Responsibility	Protection
Request threads	Uvicorn	Serve HTTP. Read rolling state, control the stream, mutate investigation cases.	Every read takes a detached snapshot, so a request never holds a lock while serialising JSON.
pseudo-stream	StreamProcessor.start	Owns the scoring loop. Pulls events from the generator, predicts, assesses risk, updates accounts, raises alerts, opens cases.	A single threading.Lock prevents two concurrent streams; a threading.Event requests a cooperative stop.
supabase-writer	SupabaseWriter	Drains a 10,000-slot queue, groups rows by table, and flushes in batches of 25 or every second.	Producer/consumer queue with a sentinel shutdown. If the queue fills, rows are dropped with a warning rather than blocking the scoring loop.

The consequence a reviewer should know

Because the stream runs as an in-process background thread, the engine must be deployed with a **single worker process**. Two workers would each hold their own independent stream state and rolling buffers, and the dashboard would see whichever worker answered the request. This is documented in the deployment guide, and section 22 explains what a multi-worker design would require.

5.3 Concurrency primitives and where each is used

Primitive	Used in	Why that choice
Generator with <code>yield</code>	<code>transaction_stream</code>	Lazy, bounded memory, one record per iteration, resumable, stoppable mid-file.
<code>threading.Event</code>	Stop signalling	Cooperative cancellation. The generator polls a callback rather than being killed mid-write.
<code>threading.Lock</code>	Stream start, token cache, singletons	Simple mutual exclusion where no nested acquisition can happen.
<code>threading.RLock</code>	Rolling state, investigation store	Re-entrant, because internal helpers call other locking methods on the same object.
Bounded <code>deque</code>	5,000 latencies, 500 transactions, 500 alerts, 120 timeline buckets, 64 account events	Ring buffers give O(1) append with automatic eviction, so memory is constant no matter how long the stream runs.
<code>queue.Queue</code>	Persistence handoff	Thread-safe producer/consumer channel with a hard capacity, which converts unbounded backlog into a measurable drop count.
Double-checked locking	Predictor, processor and writer singletons	The model artifacts are deserialised exactly once per process without paying lock cost on every access.
Optimistic concurrency	Investigation case mutations	Version numbers instead of locks held across HTTP requests, so two analysts cannot silently overwrite each other.
Welford's online algorithm	Per-account amount mean and variance	Streaming statistics in constant memory, numerically stabler than accumulating a sum of squares.

6 End-to-end data flow

6.1 The life of one transaction

This is the sequence that runs for every single record. Steps 2 to 4 are the timed section that the 50 ms budget applies to; everything after step 7 happens off the critical path.

THE ELEVEN STEPS, IN EXECUTION ORDER

#	Step	What happens	Implementation detail
1	Receive and validate	The generator reads one CSV row, checks that Time and Amount exist, are numeric and non-negative, and that all 28 PCA components parse.	A malformed row raises an internal error that is caught, counted and skipped. The stream continues; it never dies on bad input.
2	Build the feature vector	28 PCA components are copied into a pre-allocated array, then five engineered values are appended: amount, log amount, seconds of day, hour of day and a night flag.	A NumPy array of exactly 33 float64 values in fixed order. No DataFrame is created.
3	Transform	Median imputation then z-score standardisation, using statistics fitted on the training split only.	The same serialised scikit-learn Pipeline object that training produced, so offline and online transforms cannot drift.
4	Predict	XGBoost forward pass returns the probability of the positive class.	This is where the timer stops. Measured average 0.9739 ms.
5	Risk assessment	The probability is rescaled so the tuned threshold lands exactly on 0.70, then mapped to a band and an action.	Piecewise linear and monotonic, so relative ordering of transactions is preserved.
6	Behavioural features	Seven behavioural values are computed from the account's <i>prior</i> activity: prior count, average amount, deviation, seconds since previous, one-hour velocity, distinct recent locations, high-value flag.	Computed before the account profile is updated, so the current transaction can never be part of its own baseline.
7	Account update	The profile absorbs the new transaction, recomputes seven weighted signals and reassigns an account risk level.	Welford update for mean and variance; bounded dequeues for recent times and locations.
8	Alert decision	If the risk score is at or above 0.70, an alert row is created with a UUID, an alert type and an open status.	Alert type is the first matching rule of six; reason codes capture <i>all</i> matching evidence.
9	Explain and open a case	For alerting events only, TreeSHAP contributions are computed and an investigation case is opened linking alert, transaction snapshot, explanation and reason codes.	Deliberately after the latency measurement, so explainability costs nothing on the measured path. Non-XGBoost models degrade to an explicit "unavailable" result.

#	Step	What happens	Implementation detail
10	Record in rolling state	Counters, risk histogram, latency window, transaction and alert ring buffers, per-second timeline bucket and the live confusion matrix all update under one lock.	Only transactions carrying a ground-truth label update the confusion matrix.
11	Hand off persistence	Rows are enqueued for the writer thread. Account rows are only sent when the account carries risk, to keep write volume sane.	Fire and forget. If Supabase is not configured, this is a no-op and the dashboard runs from memory.

6.2 How the dashboard stays current

Two independent mechanisms feed the console, which is why it still works when the database is not configured.

Mechanism 1: polling the engine

A single interval in `StreamContext` fetches six endpoints per tick. The cadence is adaptive: **no timer at all** while the first load is in flight, **2000 ms** while a stream is running, and **8000 ms** when idle. Pausing the feed skips the tick without destroying the timer. This path needs only the API.

Mechanism 2: Realtime push

One websocket channel, `fraudstream-live`, subscribes to five table events: transaction inserts, alert inserts, any account risk change, any investigation case change and note inserts. Rows are merged by primary key, so a row that arrives by both push and poll can never appear twice. Requires Supabase.

WHAT EACH POLL TICK ACTUALLY REQUESTS

Endpoint	Limit	Feeds
<code>GET /api/metrics</code>	—	Every KPI tile, both charts, risk distribution, live precision and recall, persistence stats, investigation counters
<code>GET /api/transactions/recent</code>	120	Overview feed preview and the live monitor table
<code>GET /api/alerts</code>	80	Alerts page, sidebar badge, overview alert panel
<code>GET /api/accounts/high-risk</code>	60, minimum level medium	Accounts page and the overview top-accounts list
<code>GET /api/investigations</code>	200	Case queue and the sidebar case badge
<code>GET /api/investigations/metrics</code>	—	The four workbench tiles

Reference data that rarely changes (`/api/health`, `/api/model`, `/api/dataset/info`) is fetched once at mount and again after a dataset upload, rather than on every tick.

6.3 Degradation behaviour

If this is missing	What happens	User-visible result
Supabase credentials	Writer becomes a no-op; Realtime is disabled; auth falls back to a development user	Dashboard runs entirely from the engine's in-memory buffers. Status pill reads "Polling only".

If this is missing	What happens	User-visible result
Trained model artifacts	Predictor raises a specific error; health reports "degraded"; stream start returns 503	A banner tells the reviewer to run the training command. The stream button is disabled with a tooltip.
Held-out stream file	Falls back to the full dataset with a warning	Streaming still works, with a logged caveat that rows may have been seen in training.
The engine itself	Axios reports a connection failure with the base URL in the message	"Engine offline" pill and an explicit error banner naming the URL it tried.
XGBoost explainability	<code>explain()</code> catches the failure and returns <code>available: false</code> with a reason	The case still opens with reason codes; the evidence panel shows an "Unavailable" badge and the stated reason.

7 The dataset and exploratory analysis

We use the ULB credit card fraud dataset (`creditcard.csv`): two days of real European card transactions from 2013. Features V1 to V28 are principal components produced by the publishers to anonymise the original fields; only `Time`, `Amount` and the `Class` label are in their original form.

7.1 Cleaning report, generated on every training run

FOUR CLEANING STEPS

Step	Rows affected
Rows read from file	284,807
Exact duplicates removed	1,081
Rows dropped for missing values	0
Rows dropped for negative Time or Amount	0
Rows used for modelling	283,726

CLASS BALANCE AFTER CLEANING

Measure	Value
Fraudulent transactions	473
Legitimate transactions	283,253
Fraud rate	0.1667 %
Negative to positive ratio	1 : 598.84
Time span covered	48.0 hours

Why the duplicates matter

Removing 1,081 exact duplicates is not cosmetic. If a duplicated row landed in both the training and the test split, the model would be tested on a record it had already memorised, and every score would be inflated. De-duplication happens **before** the split for exactly that reason.

7.2 Amount distribution

Statistic	All transactions	Fraud only	Legitimate only
Mean amount	88.47	123.87	88.41
Median amount	22.00	—	—
95th percentile	365.34	—	—
99th percentile	1,018.97	—	—
Maximum	25,691.16	—	—

The median is 22.00 while the maximum is 25,691.16, so the distribution has a very long right tail. That is why we add `log_amount = ln(1 + amount)` as a feature: it compresses the tail so a single large purchase does not dominate the scaled feature space, and `log1p` stays defined at an amount of zero.

7.3 Fraud concentrates in quiet hours

Grouping by hour of day within the 48-hour window shows a clear pattern, and it is the direct justification for the `hour_of_day` and `is_night` features.

SELECTED HOURS, SHOWING FRAUD DENSITY RATHER THAN FRAUD COUNT

Hour	Transactions	Fraud	Fraud rate	Reading
02:00	3,308	48	1.451 %	Lowest volume window, highest fraud density: roughly 8.7 times the overall rate

Hour	Transactions	Fraud	Fraud rate	Reading
04:00	2,204	23	1.044 %	Second densest, again in the quiet hours
03:00	3,487	17	0.488 %	Still well above the baseline
11:00	16,781	53	0.316 %	Highest absolute fraud count, but a much lower rate because volume is five times higher
10:00	16,548	8	0.048 %	Busy and clean: the safest hour in the sample
Overall	283,726	473	0.167 %	Baseline for comparison

The analytical point worth making to a panel

Hour 11 has the most frauds in absolute terms, but hour 02 is roughly **4.6 times more dangerous per transaction**. Counting alerts would point an investigator at the busy middle of the day; measuring density points them at the small hours. This is the same distinction as accuracy versus precision-recall, applied to time.

7.4 Which features carry signal

TOP TEN ABSOLUTE CORRELATIONS WITH THE FRAUD LABEL

Rank	Feature	correlation	Rank	Feature	correlation
1	V17	0.3135	6	V3	0.1823
2	V14	0.2934	7	V7	0.1723
3	V12	0.2507	8	V11	0.1491
4	V10	0.2070	9	V4	0.1293
5	V16	0.1872	10	V18	0.1053

No single feature is close to decisive; the strongest linear relationship is 0.31. Fraud here is a **combination** of weak signals, which is precisely the situation where tree ensembles outperform linear models, and it is also why the TreeSHAP explanation in section 15 typically shows several features contributing together rather than one dominant cause.

7.5 The chronological split

Split	Rows	Fraud	Fraud rate	Time window (seconds from first transaction)
Train (70 %)	198,608	366	0.184 %	0 to 132,906
Validation (15 %)	42,558	55	0.129 %	132,906 to 151,320
Test (15 %)	42,560	52	0.122 %	151,320 to 172,792

Sorting uses a stable merge sort, so transactions sharing a timestamp keep their original relative order. The test split is written out as `data/stream_test.csv` and becomes the replay stream, which means the live demo is scoring **data the model has never seen**. A companion index file records that the first labelled fraud is at row 1,326 and recommends starting at row 1,311, so a demonstration reaches a real alert in seconds instead of minutes.

8 Machine learning pipeline

8.1 Feature set: 33 inputs, and what is deliberately excluded

THE FIVE ENGINEERED FEATURES

Feature	Definition
amount	Transaction value, clipped at zero
log_amount	$\ln(1 + \text{amount})$, tail compression
seconds_of_day	Time mod 86400
hour_of_day	$\text{floor}(\text{seconds_of_day} / 3600)$
is_night	1 when hour is 0 to 5, else 0

EXCLUDED FROM THE MODEL ON PURPOSE

Attribute	Reason
account_id	All six are derived from a hash of the PCA values, because the anonymised dataset has no such fields. Feeding them to the model would be circular reasoning. They exist only for aggregation and display. A unit test asserts that none of these names appears in the model feature list.
card_last4	
merchant	
merchant_category	
location	
channel	

Total model input: **28 PCA components + 5 engineered = 33 features**, always in the same order, defined once in the shared configuration module. Preprocessing is a two-step scikit-learn Pipeline: median imputation, then standardisation to zero mean and unit variance. It is fitted on the training split only and serialised alongside the model, so the exact same statistics are applied at inference time.

8.2 Threshold tuning: why the cut-off is 0.184152, not 0.5

A default threshold of 0.5 assumes both errors cost the same and both classes are equally common. Neither is true here. A missed fraud costs the transaction value; a false alarm costs a review. Our tuning procedure runs on the validation split only:

1. Sweep the entire precision-recall curve.
2. Discard every threshold where precision falls below **0.50**, so at least half of all alerts must be real.
3. Among those that survive, maximise **F2**, which weights recall four times more heavily than precision.
4. Clamp the result strictly inside (0, 1), because a threshold of exactly 1.0 would only fire on a probability that is bit-for-bit 1.0.

0.184152 CHOSEN THRESHOLD Strategy: max F2 with precision floor	0.8824 PRECISION AT THRESHOLD Validation split	0.8182 RECALL AT THRESHOLD Validation split	0.8303 F2 AT THRESHOLD The value being maximised
---	--	--	---

8.3 Model selection: candidate comparison

ALL FOUR CANDIDATES ON THE VALIDATION SPLIT, WITH MEASURED SINGLE-TRANSACTION LATENCY

Candidate	PR-AUC	ROC-AUC	Recall	F1	p95 latency	Fit time	Outcome
Random Forest	0.8715	0.9783	0.8182	0.8738	33.16 ms	44.05 s	Rejected on latency
XGBoost	0.8472	0.9880	0.8182	0.8491	1.76 ms	4.85 s	Selected
Logistic Regression	0.8414	0.9817	0.7818	0.8113	2.23 ms	1.09 s	Baseline, lower quality

Candidate	PR-AUC	ROC-AUC	Recall	F1	p95 latency	Fit time	Outcome
Isolation Forest	0.0553	0.9541	0.5273	0.1408	6.13 ms	1.97 s	Unusable precision

The most interesting decision in the project

Random Forest had the best PR-AUC, and a naive selection rule would have shipped it. It was rejected because its p95 latency of **33.16 ms** consumes two thirds of the entire 50 ms budget on an idle development laptop, leaving nothing for a loaded production host. The selection rule is therefore three-stage: a hard gate at 50 ms, a comfort filter at 25 ms which demands two times headroom, and only then the best PR-AUC. XGBoost gives up 0.024 PR-AUC and gains a **19-fold** latency improvement. The log line recording that trade-off is printed by the training run itself.

8.4 Final scorecard, including F1 and accuracy

These are the numbers a panel usually asks for first. Both splits are shown, at the single tuned threshold of 0.184152. Accuracy is computed from the confusion matrices below; note carefully how little it tells you.

COMPLETE METRIC SCORECARD FOR THE SHIPPED MODEL, XGBOOST V1.0.0

Metric	Validation	Test (held out)	What it means
F1 score	0.8491	0.7500	Harmonic mean of precision and recall. On test, precision and recall are both exactly 0.75, so F1 equals 0.75.
Accuracy	99.9624 %	99.9389 %	Share of all transactions classified correctly: 42,534 of 42,560 on test. Impressive looking and almost meaningless here, see the note below.
Precision	0.8824	0.7500	Of everything we flagged, this fraction really was fraud.
Recall (sensitivity)	0.8182	0.7500	Of all actual fraud, this fraction was caught.
PR-AUC (average precision)	0.8472	0.7629	Threshold-independent quality on the rare class. Our headline metric.
ROC-AUC	0.9880	0.9737	Ranking quality overall. Optimistic under heavy imbalance, so reported but not relied on.
False positive rate	0.000141	0.000306	0.03 % of legitimate transactions were disturbed.
Specificity	0.999859	0.999694	Correctly cleared share of legitimate traffic.
Balanced accuracy	90.90 %	87.48 %	Mean of recall and specificity. The honest counterpart to raw accuracy.
Matthews correlation	0.8495	0.7497	Single-figure quality that accounts for all four matrix cells; robust to imbalance.

CONFUSION MATRIX, VALIDATION (42,558 ROWS, 55 FRAUD)

	Predicted fraud	Predicted legitimate
Actual fraud	45 TP	10 FN
Actual legitimate	6 FP	42,497 TN

CONFUSION MATRIX, TEST (42,560 ROWS, 52 FRAUD)

	Predicted fraud	Predicted legitimate
Actual fraud	39 TP	13 FN
Actual legitimate	13 FP	42,495 TN

Read this before quoting the accuracy figure

Our test accuracy is **99.9389 %**. Now consider a model consisting of one line of code: `return "legitimate"` . On the same test split that model scores **99.8778 %** accuracy, because only 52 of 42,560 rows are fraud. It catches zero fraud and is worthless, yet it is only **0.061 percentage points** behind us.

That is why the codebase never reports accuracy anywhere: not in the training summary, not in the metadata, not on the analytics page. We include it in this report only because reviewers ask for it, and always beside **F1 0.75**, **PR-AUC 0.7629** and **balanced accuracy 87.48 %**, which actually distinguish a working detector from a constant function.

8.5 Why validation scores exceed test scores

F1 falls from 0.8491 to 0.7500 between validation and test. We are not hiding that, and there is a straightforward explanation:

- The threshold was tuned on validation.** Any threshold chosen to optimise a split will flatter that split. Test is the only untouched measurement, so 0.75 is the number we stand behind.
- Tiny positive counts.** Test contains 52 frauds. Moving three transactions across the boundary shifts recall by nearly six percentage points. Confidence intervals at this sample size are wide, and section 22 states that plainly.
- Time moves forward.** Test is the latest window in the timeline. A drop from validation to test is the expected signature of mild distribution drift, which is exactly what a chronological split is designed to expose rather than hide.

8.6 Latency benchmark method

Latency is measured through the identical three calls the live server makes: build the feature vector, transform it, run the forward pass. Twenty records are scored untimed as warm-up, then 500 records are timed individually with a high-resolution counter, sorted, and reduced to interpolated percentiles.

Average	Median	p95	p99	Min	Max	Target	Verdict
0.9739 ms	0.9185 ms	1.5880 ms	1.9320 ms	0.5453 ms	2.2913 ms	50 ms	Pass, 31x inside budget

8.7 Artifacts the pipeline produces

Artifact	Contents
<code>models/fraud_model.joblib</code>	The fitted XGBoost classifier
<code>models/preprocessor.joblib</code>	The fitted imputer and scaler pipeline
<code>models/model_metadata.json</code>	Threshold and tuning strategy, risk bands, feature names, excluded fields, both metric sets, both confusion matrices, the precision-recall curve, latency statistics, every candidate's results, the selection criteria string and the leakage check result

Artifact	Contents
data/stream_test.csv	The held-out test split, ready to replay as a stream
data/dataset_profile.json	The full EDA profile that the dashboard reads
data/stream_fraud_index.json	Where labelled fraud sits in the stream file, plus the densest 400-row window

Because the metadata carries the threshold, the risk bands and the metrics, the API can serve the model card without re-reading the dataset, and the dashboard's analytics page is generated from the same file the trainer wrote. Retraining updates the dashboard automatically.

9 Risk engine mathematics

A raw probability is a poor operational signal: 0.184152 means "flag this" for our model, but that number is meaningless to an analyst and would change on retraining. The risk engine converts probability into a stable, human-readable 0 to 1 risk score with fixed action bands.

9.1 Probability to risk score

```
# Piecewise linear, monotonic, anchored on the tuned threshold
if probability <= threshold:
    score = 0.70 * (probability / threshold)
else:
    score = 0.70 + (1.0 - 0.70) * ((probability - threshold) / (1.0 - threshold))
```

Property	Consequence
probability 0 maps to 0.00	Clean transactions sit at the bottom of the scale.
probability = threshold maps to exactly 0.70	The alert boundary is a fixed constant, 0.70, no matter what the model's threshold becomes after retraining.
probability 1 maps to 1.00	The scale is fully used.
Strictly monotonic	The relative ordering of transactions by risk is never altered, so no ranking information is lost in the conversion. A 101-step sweep test asserts this.

9.2 The four risk bands

Band	Range	Action	Operational meaning
Low	0.00 to 0.39	Allow	Approve silently. The overwhelming majority of traffic.
Medium	0.40 to 0.69	Monitor	Approve but count towards account risk.
High	0.70 to 0.89	Flag	Raise an alert and open an investigation case.
Critical	0.90 to 1.00	Alert and investigate	Alert with urgent case priority.

An alert is raised at a risk score of **0.70 or above**, which by construction is the same as "the model probability reached the tuned threshold". The two rules cannot drift apart because the mapping anchors them together.

9.3 Account-level risk: seven weighted signals

Single transactions are only half the picture. Fraudsters test a stolen card with small purchases before a large one, so the engine maintains a rolling profile per account and combines seven normalised signals.

SIGNAL WEIGHTS SUM TO EXACTLY 1.00, ASSERTED BY A UNIT TEST

Signal	Weight	Formula, clamped to 0 to 1	What it detects
Average risk	0.30	Mean transaction risk for the account	Sustained elevated risk rather than one outlier
Suspicious ratio	0.20	suspicious count / transaction count	Proportion of activity that alerted

Signal	Weight	Formula, clamped to 0 to 1	What it detects
Maximum risk	0.15	Highest single transaction risk seen	The worst moment on the account
Velocity	0.10	(events in the last hour – 1) / 5, saturating at 6 events	Rapid-fire card testing
High value ratio	0.10	transactions above 500 / total	Concentration of large purchases
Geographic spread	0.08	(distinct recent locations – 1) / 2, saturating at 3	Impossible travel patterns
Merchant anomaly	0.07	most frequent suspicious category count / 3	Repeated abuse of one merchant type

The suspicion floor

After the weighted sum, one override applies: if an account has produced at least one suspicious transaction, its risk score is floored at **0.70**. Without it, an account with 400 clean transactions and one confirmed critical hit would average down into the low band and vanish from the queue. A long clean history must not be able to dilute a confirmed hit. The consequence is stated on the accounts page in the interface itself.

9.4 Behavioural features are computed before the update

Ordering matters here and is easy to get wrong. For each transaction the engine first reads the account's prior state to build behavioural features, and only then folds the new transaction into the profile. If the order were reversed, the transaction would appear in its own baseline, its amount deviation would be biased towards zero, and its velocity count would always be at least one. A dedicated test asserts prior-activity-only behaviour, and the transaction detail drawer tells the analyst the same thing.

Behavioural feature	Definition
Prior transaction count	How many transactions this account had before this one
Average prior amount	Running mean maintained by Welford's algorithm
Amount deviation	Z-score against the account's own history, with a relative fallback below two samples
Seconds since previous	Gap to the last event, or null on first sighting
One-hour velocity	Events within the preceding 3,600 seconds
Distinct recent locations	Unique locations across the last 64 events
High value flag	Amount at or above 500

9.5 Reason codes: evidence in words

Every alert carries reason codes. Unlike the single legacy alert label, reason codes report **all** matching evidence, each with the observed value and the threshold it crossed, so an analyst can audit the trigger without reading code.

Code	Category	Fires when	Analyst-facing label
<code>critical_model_risk</code>	Model	Risk band is critical	Critical model risk
<code>high_model_risk</code>	Model	Risk band is high	High model risk

Code	Category	Fires when	Analyst-facing label
transaction_velocity	Behaviour	6 or more events in an hour	Rapid transaction velocity
high_value_amount	Behaviour	Amount at or above 500	High-value transaction
amount_deviation	Behaviour	Z-score at or above 3.0	Amount deviates from account history
geographical_anomaly	Behaviour	3 or more distinct recent locations	Multiple recent locations
repeat_account_activity	Account	3 or more prior transactions	Established account pattern

10 Backend code walkthrough

Every Python module, in dependency order, with its size, its single responsibility, and the concepts a reader needs in order to follow it. 6,506 lines across 37 files.

10.1 Shared foundation

Module	Lines	Responsibility and key contents	Concepts used
<code>src/config.py</code>	238	The single source of truth. Defines the 28 PCA names, the 5 engineered names, the combined 33-feature order, the four risk bands, the alert threshold 0.70, the 50 ms latency target, the training configuration (70/15/15, seed 42, precision floor 0.50, F-beta 2.0), all artifact paths and every environment-variable-backed setting. Also resolves the dataset by searching three candidate locations.	Frozen dataclasses for immutability, a settings object read once, environment overrides, tuple constants so ordering cannot be mutated at runtime

10.2 Offline pipeline modules

Module	Lines	Responsibility and key functions	Concepts used
<code>preprocessing/loader.py</code>	192	<code>load_dataset</code> validates the schema and raises actionable errors naming the missing columns. <code>clean_transactions</code> runs the four cleaning steps and returns the report shown in section 7. <code>time_aware_split</code> sorts by time with a stable merge sort and slices 70/15/15. <code>leakage_check</code> refuses any feature named like a label.	Typed CSV reading with float32 for PCA columns to save memory, stable sorting, dataclass split container, defensive validation
<code>preprocessing/preprocess.py</code>	52	Builds the two-step Pipeline (median imputer then standard scaler), fits it on training data only, and selects the 33 feature columns in canonical order.	scikit-learn Pipeline so fitted statistics travel inside the artifact, median rather than mean imputation for tail robustness
<code>features/engineering.py</code>	84	Two numerically identical implementations of the five engineered features: a vectorised one for training and a scalar one for streaming. <code>feature_vector</code> fills a pre-allocated 33-slot array and tolerates missing components by defaulting them to 0.0.	Deliberate code duplication with test-enforced parity, pre-allocation instead of per-record DataFrames, graceful degradation on malformed input

Module	Lines	Responsibility and key functions	Concepts used
features/identity.py	119	Derives account, card suffix, merchant, category, location and channel from quantised PCA values using a BLAKE2b digest. 420 accounts, 10 categories, 20 locations, 4 channels.	Cryptographic hashing for determinism across machines (Python's built-in hash is salted per process), quantisation so similar rows collide onto the same account, bit-shifting to derive independent values from one seed
training/evaluate.py	221	<code>classification_metrics</code> computes precision, recall, F1, FPR, PR-AUC, ROC-AUC and the confusion matrix, with explicit labels so a degenerate prediction cannot produce a malformed matrix. <code>tune_threshold</code> implements the F2-with-floor strategy. <code>latency_benchmark</code> warms up then times individual records. <code>pr_curve_points</code> downsamples the curve to 60 points for the chart.	Imbalance-aware metric selection, F-beta mathematics with divide-by-zero suppression, interpolated percentiles, no accuracy anywhere
training/train.py	709	The orchestrator. Eight stages: load, clean, split and write the stream file, engineer features and check leakage, fit the preprocessor, train and evaluate every candidate, select the winner with the latency gate, then serialise artifacts and print a comparison table. Also builds the EDA profile and optionally pushes metrics to the database.	Candidate registry pattern, optional dependency guarding so a missing XGBoost falls back rather than crashing, satisficing versus maximising selection logic, memory release between stages, a CLI with fast mode and model filters
inference/probability.py	61	One uniform way to extract P(fraud). Supervised models use <code>predict_proba</code> column 1; unsupervised detectors have their decision function inverted and squashed through a calibrated sigmoid.	Numerically stable sigmoid that avoids overflow on large negative inputs, median and standard deviation calibration, single extraction point so training and serving cannot diverge
inference/predictor.py	245	Loads the artifacts once, warms up with a zero record so the first real request is not an outlier, and exposes <code>predict</code> (the timed path) and <code>explain</code> (TreeSHAP, off the timed path). Raises a specific error naming the training command when artifacts are missing.	Process-wide singleton with double-checked locking, explicit warm-up, timing scoped to exactly the work being claimed, graceful explanation fallback

10.3 Streaming and risk modules

Module	Lines	Responsibility and key functions	Concepts used
<code>streaming/generator.py</code>	189	The event source. Reads the CSV with the standard library, validates each row, derives identity, timestamps the event against a fixed epoch, and yields one <code>TransactionEvent</code> . Supports limit, skip, inter-arrival delay, a stop callback and an invalid-row callback.	Generator with <code>yield</code> , cooperative cancellation polled per row, error isolation so one bad row cannot end the stream, the label deliberately excluded from the model payload
<code>streaming/state.py</code>	345	All rolling state: status, config, counters, risk histogram, latency window of 5,000, transaction and alert buffers of 500 each, 120 one-second timeline buckets and the live confusion matrix. Computes latency percentiles, throughput, risk distribution and live precision and recall.	Re-entrant lock guarding every access, bounded dequeues as ring buffers, snapshot copies returned to callers, statistics computed outside the lock to shorten the critical section
<code>streaming/processor.py</code>	489	Owens the lifecycle and the eleven-step per-event pipeline from section 6. Resolves the stream source safely, refuses concurrent runs, spawns the worker thread, builds the dashboard row and alert row contracts, and assembles the single <code>/api/metrics</code> payload.	Background daemon thread, dependency injection of predictor and writer for testability, lazy singletons, defensive terminal-state handling that distinguishes a requested stop from completion from failure
<code>streaming/index.py</code>	129	Scans the stream file for labelled fraud positions and computes the densest 400-row window with a two-pointer sweep, so a demonstration can start near real fraud without reordering anything.	Sliding window algorithm, streaming file scan that never loads the file into memory, honest documentation that only the offset changes
<code>risk/scoring.py</code>	489	Everything in section 9: the probability mapping, band classification, the account engine with seven weighted signals, behavioural feature extraction, the suspicion floor, reason codes and the legacy alert label.	Welford's online variance, saturating normalisation of unbounded counts, weight vector that must sum to one, clamping helper that also absorbs NaN
<code>investigations/store.py</code>	445	The case workflow: create from alert with duplicate suppression, list with filters, assign, add notes, transition status, resolve with an outcome code, compute workflow metrics, and flatten cases for persistence. Evicts the oldest cases beyond a 2,000-case ceiling.	Optimistic concurrency with version checks, deep-copy snapshots so callers cannot mutate internal state, append-only event log per case, ground-truth label stripped from the snapshot so it cannot bias an analyst

10.4 Persistence and API modules

Module	Lines	Responsibility and key functions	Concepts used
<code>persistence/supabase_client.py</code>	224	Buffered background writer. Six write methods enqueue rows; the consumer groups them by table and conflict key, deduplicates upserts, and flushes at 25 rows or every second. Exposes counters for succeeded, failed, dropped and last error.	Producer/consumer queue with a hard 10,000 capacity, sentinel-based shutdown, dual size and time batching, upsert deduplication because Postgres rejects a command touching the same conflict row twice, load shedding instead of back-pressure
<code>api/auth.py</code>	130	Verifies the bearer token against the Supabase user endpoint and caches the result for 60 seconds, bounded at 512 entries. Returns a development user when authentication is not fully configured, warning exactly once.	FastAPI dependency injection, TTL cache to avoid an auth round trip on every 2-second poll, distinct 401 and 503 handling so an outage is not reported as a bad credential
<code>api/schemas.py</code>	159	Seven Pydantic models. Notably the stream request forbids unknown fields and runs the path-traversal validator; the investigation schemas all carry an optional <code>expected_version</code> ; note and summary bodies enforce length limits and reject whitespace-only text.	Declarative validation at the edge, regex-constrained enumerations, extra-field policies chosen per schema, custom field validators
<code>api/main.py</code>	643	21 routes, CORS configuration, the chunked upload handler with a 250 MB ceiling enforced mid-stream, and startup and shutdown hooks that preload the model and drain the writer.	Router prefixing, literal routes declared before parameterised ones so <code>/metrics</code> is not swallowed by <code>/case_id</code> , streaming upload with partial-file cleanup, HTTP status codes chosen to be diagnostic rather than generic
<code>scripts/smoke_e2e.py</code>	374	Spawns a real server, then asserts 23 end-to-end checks including a concurrent-start conflict, latency inside budget, risk distribution summing to the transaction total, alert triage, account signal completeness, and the full upload and stream cycle with cleanup.	Black-box integration testing over HTTP, polling with timeouts, guaranteed teardown, test artifacts removed afterwards

Two behaviours we disclose rather than hide

1. Authentication fails open. Enforcement requires the `require-auth` flag *and* a project URL *and* an anon key. Remove any one and every protected route silently accepts requests as a development user, with a single log warning. This makes local development frictionless and is documented in the code, but it means a misconfigured deployment is unauthenticated. Section 16 gives the recommended fix.

2. Persisted counters read zero in stream status. The stream status payload always reports zero persisted rows because the final bookkeeping call passes zeros. Real persistence counts are available under the metrics payload's persistence block. It is a reporting seam, not data loss.

11 API reference

Base URL `http://localhost:8000`, everything under `/api`, interactive docs at `/docs`. Only health and the service root are public; every other route requires `Authorization: Bearer <supabase access token>`.

COMPLETE ENDPOINT LIST

Method	Path	Auth	Purpose	Errors
GET	<code>/api/health</code>	public	Readiness: model loaded, dataset present, stream source present, Supabase configured, auth enforced, uptime	—
GET	<code>/</code>	public	Service name, version and doc links	—
POST	<code>/api/predict</code>	required	Score one ad-hoc transaction; optionally also fold it into account risk	422, 503
POST	<code>/api/stream/start</code>	required	Start the replay with source, limit, delay, skip, persist and reset options	409, 400, 422, 503
POST	<code>/api/stream/stop</code>	required	Request a cooperative stop with a five second grace period	—
GET	<code>/api/stream/status</code>	required	State machine value, progress, elapsed time, throughput, error text	—
GET	<code>/api/metrics</code>	required	The whole dashboard in one payload: nine top-level blocks	—
GET	<code>/api/transactions/recent</code>	required	Newest scored transactions, optional risk band filter	422
GET	<code>/api/alerts</code>	required	Buffered alerts, optional high or critical filter	422
PATCH	<code>/api/alerts/{transaction_id}</code>	required	Triage an alert to open, investigating, resolved or dismissed	404, 422
GET	<code>/api/investigations</code>	required	Case queue with status, risk, assignee, unassigned, limit and offset filters	422
GET	<code>/api/investigations/metrics</code>	required	Workflow analytics: counts by status, resolution mix, confirmation rate, mean time to resolve	—
GET	<code>/api/investigations/{case_id}</code>	required	Full case: evidence, explanation, reason codes, notes, event log	404
PATCH	<code>/api/investigations/{case_id}/assignment</code>	required	Assign or unassign an owner	404, 409, 422

Method	Path	Auth	Purpose	Errors
POST	/api/investigations/{case_id}/notes	required	Append an analyst note, up to 4,000 characters	404, 409, 422
PATCH	/api/investigations/{case_id}/status	required	Transition workflow state and mirror onto the alert	404, 409, 422
POST	/api/investigations/{case_id}/resolution	required	Close with an outcome code, rationale and confidence	404, 409, 422
GET	/api/accounts/high-risk	required	Escalated accounts ranked by risk, with the seven signal values	422
GET	/api/accounts/{account_id}	required	One account profile plus its recent transactions	404
GET	/api/model	required	The model card: metrics, candidates, threshold tuning, bands, leakage check, live quality	503
GET	/api/dataset/info	required	Dataset paths and sizes, EDA profile, fraud index, uploaded files	—
POST	/api/dataset/upload	required	Upload a CSV up to 250 MB into the uploads directory	400, 413

11.1 Status codes chosen to be diagnostic

Code	When	Why not something else
409	A stream is already running	Returning 200 for "did nothing" makes a caller poll a stream it did not start. 409 states exactly what happened.
409	Case version mismatch	Tells the browser its copy is stale so it can refresh, instead of silently overwriting another analyst's work.
413	Upload above 250 MB	Detected mid-stream, so the server never buffers a huge file before rejecting it, and the partial file is deleted.
422	Schema or path-traversal violation	Validation happens before any handler code runs, so malformed input cannot reach business logic.
503	Model not trained, or auth service unreachable	Distinguishes "this service is not ready" from "your credentials are wrong", which would be 401.

11.2 The path-traversal guard, in detail

The stream endpoint accepts a source file name. Without validation, a request could name any readable CSV on the host and stream its contents into the dashboard. The validator normalises backslashes to forward slashes, then requires the value to equal its own final path segment, and finally requires a `.csv` suffix. Six hostile inputs are asserted to fail in the test suite:

Rejected input	Attack it represents
<code>../creditcard.csv</code>	Relative escape one level up

Rejected input	Attack it represents
<code>..\..\windows\system32\drivers\etc\hosts</code>	Windows-style traversal to a system file
<code>/etc/passwd</code>	Absolute POSIX path
<code>C:\Windows\win.ini</code>	Absolute Windows path
<code>uploads/../../secrets.csv</code>	Traversal disguised inside a legitimate-looking prefix
<code>notes.txt</code>	Wrong file type, blocked even without traversal

12 Database layer

Two idempotent migrations, 475 lines of SQL, seven tables. Both can be re-run safely because every object creation is guarded and every trigger and policy is dropped before being recreated.

12.1 Tables

Table	Migration	Purpose and notable columns	Indexes
profiles	0001	One row per user, linked to the auth system, with a role constrained to analyst, admin, developer or viewer. Populated automatically by a trigger on user creation.	Primary key only
transactions	0001	22 columns: identity, amount, merchant context, both scores, band, decision, both latencies, the dataset label kept for offline evaluation only, and behavioural context as JSON.	Created-at descending, account, risk level, and a partial index on flagged rows only
fraud_alerts	0001 + 0002	Alerts at or above 0.70, with a status lifecycle and an updated-at trigger. Migration 0002 adds a nullable case link with a partial unique index.	Created-at, status, account, transaction, plus the case link
account_risk	0001	Rolling account aggregates, upserted on account id: counts, average and maximum risk, current score and band, last activity.	Risk score descending, risk level
model_metrics	0001	One row per training run: precision, recall, F1, PR-AUC, ROC-AUC, average latency and threshold, so model quality has a history.	Created-at descending
investigation_cases	0002	The workbench: case number, alert and transaction links, priority, status, assignee, immutable transaction and alert snapshots, the explanation and reason codes as JSON, resolution code, rationale, analyst confidence, and the concurrency version.	Six: created-at, status with updated-at, priority, assignee with status, account, transaction
investigation_notes	0002	Append-only analyst notes with author attribution and a length-checked body. The only true foreign key in the schema, cascading on case deletion.	Case with created-at, author with created-at

Why alerts and transactions are linked without a foreign key

The background writer flushes rows in independent batches, so an alert can reach the database before its transaction. A hard foreign key would make write ordering matter and would turn a harmless race into a failed insert. The link is documented in a column comment instead, and the application never depends on referential order. The one place a real foreign key is correct is notes to cases, where cascade deletion is genuinely wanted.

12.2 Security model in the database

Role	Can read	Can write
<code>anon</code> NOT SIGNED IN	Nothing No policy targets this role and select is not granted.	Nothing
<code>authenticated</code> SIGNED-IN ANALYST	All detection tables, both investigation tables, and only their own profile row.	One column The alert status column, via a column-level grant, plus their own profile row. Nothing else.
<code>service_role</code> THE ENGINE	Everything; bypasses row level security.	Everything. This key exists only in the server environment file and never in the browser bundle.

The detail that shows real security thinking

Row level security can restrict which *rows* a user touches, but not which *columns*. An analyst needs to change an alert's status, and must never be able to change its risk score. So the migration revokes update on the whole alerts table and re-grants update on the status column alone. A compromised browser token can move an alert to "dismissed"; it cannot rewrite the score that raised it. When migration 0002 added the case link column, it explicitly revoked update on that column too, so the new field did not inherit the older grant.

The investigation tables go further and are **read-only for the browser**. All case mutations must pass through the API, which verifies identity, validates transitions, applies the version check and records who did what. Granting direct table writes would have bypassed all four of those guarantees.

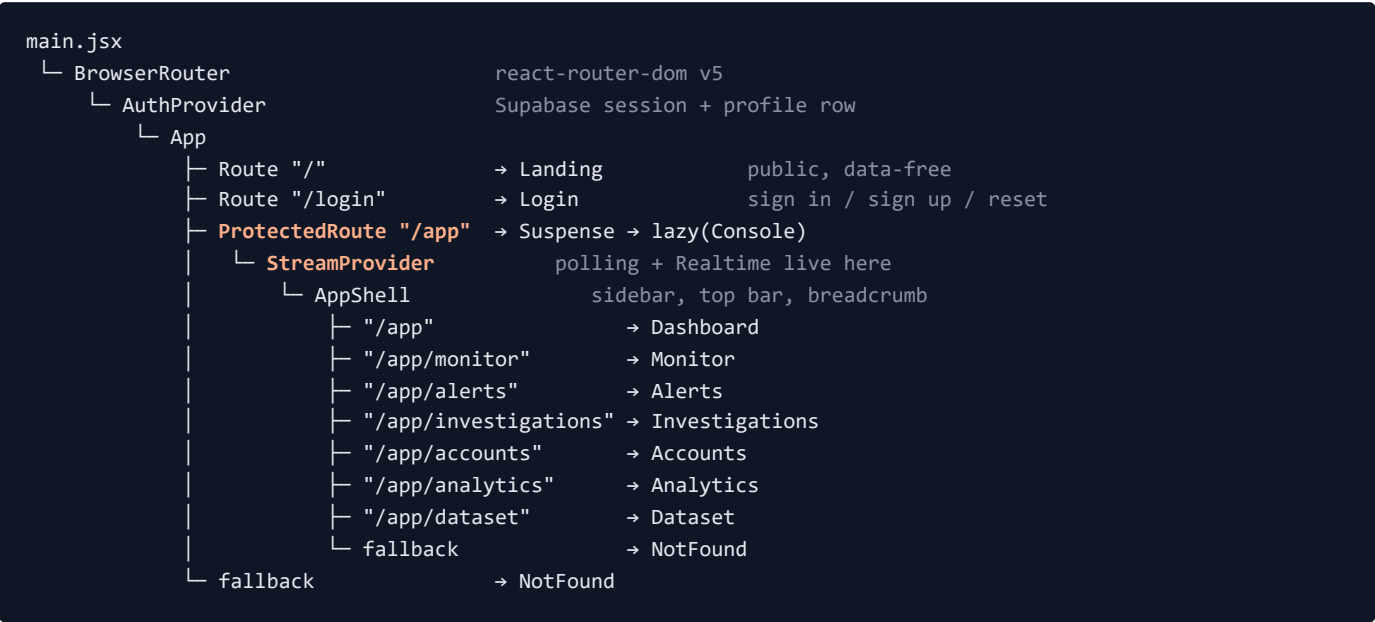
12.3 Realtime configuration

Five tables are published for live replication: transactions, alerts, account risk, investigation cases and investigation notes. Each is set to full replica identity so update events carry the complete row rather than only changed keys. The publication is created only if absent and each table is checked before being added, which is what makes the migration re-runnable. Realtime respects row level security, so a subscriber receives only rows it would have been allowed to select.

13 Frontend architecture

6,167 lines across 38 files: a public marketing site that loads without any data, and an authenticated console whose entire live state lives in one provider.

13.1 Component tree and routing



Architectural choice	Reasoning
Console is lazy loaded	Charts and live tables are the heaviest code in the app. A visitor to the marketing page never downloads them. The build produces a separate console chunk of roughly 504 kB alongside a 319 kB main chunk.
StreamProvider sits inside the guard	Polling and the websocket subscription only ever run for signed-in users, and they survive navigation between console pages instead of restarting on every route change.
AuthProvider sits above the router	Session state must outlive route changes, and the guard needs it before any route renders.
Guard waits on a loading flag	Without it, a page refresh would briefly redirect an authenticated user to the login screen while the session is restored from storage. The guard shows a restoring state instead.
Requested path is remembered	An unauthenticated visit to a deep link carries the intended destination through the redirect, and sign-in returns the user there rather than to a generic landing page.

13.2 State management: one provider, two feeds

Concern	Implementation
Adaptive polling	No timer while initialising, 2,000 ms while a stream runs, 8,000 ms when idle. Pausing skips the tick without destroying the timer.
Realtime merge	Rows arriving by websocket are merged with polled rows by primary key, so the same transaction can never appear twice.
Additive failure isolation	Investigation polling swallows its own errors on purpose, so a temporarily unavailable case route can never take down the transaction dashboard.

Concern	Implementation
Optimistic update	Alert triage patches local state immediately, then calls the API. The next poll reconciles if the server disagreed.
Optimistic concurrency	Case mutations send the version they were rendered from; a mismatch returns 409 and the drawer shows the conflict rather than clobbering another analyst.
Unmount safety	A mounted ref gates every asynchronous setter, the websocket channel is removed on cleanup, the auth listener is unsubscribed, timers are cleared and document titles are restored.
Degraded mode	When Supabase is absent the client is null, the status pill reads "Polling only", and every consumer branches on that single null check.

13.3 The seven console pages

Page	Lines	What it shows	Chart
Overview	294	Seven KPI tiles, throughput over time, risk band distribution, latency panel, live precision and recall from the streaming confusion matrix, top five accounts, latest eight transactions, six newest alerts	Area chart: transactions and flagged per second
Live monitor	166	The full 120-row buffered feed with risk band tabs, free-text search across five fields, a pause control, and a detail drawer per transaction	Line chart: average latency per second
Fraud alerts	142	Triage queue with status and severity filters, inline status editing, and a breakdown of the top five alert reasons	—
Investigations	579	Case queue plus the evidence drawer: TreeSHAP contributions, reason codes, transaction snapshot, assignment, notes, status control, resolution form and the audit trail	— (contribution meters)
High-risk accounts	131	Escalated accounts with expandable weighted signal bars, and the seven weights documented on screen	—
Model analytics	203	The model card: test and validation metrics, threshold tuning strategy, candidate comparison, two confusion matrices (test split and current stream), leakage check, latency panel	Line chart: precision-recall curve
Dataset and stream	314	Provenance, cleaning report, split sizes, top correlations, CSV upload with client-side validation, and the endpoint reference table	Bar chart: transactions and fraud by hour

13.4 Design system

CUSTOM THEME TOKENS

Token	Value
-------	-------

HAND-WRITTEN COMPONENT CLASSES

Class family	Purpose
--------------	---------

Token	Value	Class family	Purpose
Brand accent	#E8582A with 8 shades	.btn and 5 variants	Buttons with a disabled state
Ink scale	#070C16 to #3A4A69	.card, .card-header	Panel container and header
Paper background	#FAFAF8	.th, .td	Table cells with hairline rules
Hairline borders	#E6E8EC	.field-label, .field-input	Form controls, styled by hand
Type faces	Inter, JetBrains Mono	.tabular	Fixed-width digits so streaming numbers do not jitter
Extra type size	2xs at 11 px	.eyebrow, .mono	Micro-typography
Shadows	card, lift, panel	.skip-link	Keyboard skip to content
Animations	row fade-in 220 ms, dot pulse 1.6 s	.scroll-thin	Slim scrollbars in dense panels

13.5 Accessibility work

Area	What was implemented
Keyboard focus	One global focus-visible rule gives every control a two-pixel brand ring, with the offset re-targeted on dark surfaces so it stays visible.
Keyboard operation	Transaction rows are focusable and respond to Enter and Space; the stream settings popover closes on Escape and on outside click.
Live announcements	A screen-reader-only live region announces stream state and processed count as it changes.
Labels	Every search box, file input and per-row status select has an explicit or screen-reader-only label. Every icon-only button carries an aria label.
Landmarks	Real header, main, nav and footer elements; four separately labelled navigation regions; one heading per page.
Colour independence	Risk is never colour alone: every badge prints its label, and every bar has its numeric score beside it.
Reduced motion	A global media query flattens all animation, and the landing preview additionally refuses to start its rotation timer.
Titles	Every route sets a document title and restores the previous one on unmount, so navigation is announced.

Known gaps, stated honestly: the transaction drawer has no focus trap or Escape handler, the filter tabs use tab roles without arrow-key roving focus, and the charts have no text alternative, although the numeric panels beside them carry the same information. Full conformance would need manual testing with assistive technology and an expert review.

14 Frontend code walkthrough

14.1 Infrastructure modules

File	Lines	Responsibility	React concepts
<code>main.jsx</code>	16	Mounts router and auth provider into the root element using the React 16 render API.	Provider composition order
<code>App.jsx</code>	39	Top-level route table splitting public pages from the lazily loaded console, with a branded loading fallback.	<code>lazy</code> plus <code>Suspense</code> for code splitting
<code>routes/ProtectedRoute.jsx</code>	40	Three-way guard: restoring, redirect with the requested path preserved, or render.	Render props, declarative redirect, router location state
<code>console/Console.jsx</code>	38	Wraps the shell and the nested route table, and is the mount point for the live data provider.	Nested routing, provider scoping
<code>context/AuthContext.jsx</code>	147	Session bootstrap and subscription, profile row fetch, sign in, sign up, password reset, sign out, and a local demo session for machines without Supabase.	Two effects with cleanup flags, four memoised callbacks, memoised context value
<code>context/StreamContext.jsx</code>	397	The live data store described in section 13.2: sixteen pieces of state, three fetchers, one adaptive timer, one websocket channel with five listeners and nine actions.	Mounted ref guard, memoised fetchers, derived flattening of the metrics payload, key-based row merging
<code>services/api.js</code>	120	24 typed endpoint wrappers with bearer-token injection, a 20 s default timeout, a 5 minute upload timeout, and error normalisation that names the unreachable base URL.	Axios interceptor, centralised error shaping so pages never parse raw responses
<code>lib/supabaseClient.js</code>	29	Single browser client plus configuration and demo-mode flags. Returns null when unconfigured, which is the whole degraded-mode strategy.	Module-level singleton, environment-driven feature flag
<code>hooks/useInterval.js</code>	19	Declarative interval where a changing callback does not restart the timer and a null delay pauses it entirely.	Ref-held callback, two effects with different dependencies
<code>hooks/useDocumentTitle.js</code>	11	Sets a per-page title and restores the previous one on unmount.	Effect cleanup restoring prior state
<code>utils/format.js</code>	111	Thirteen formatters built on reusable Intl instances, all returning a dash placeholder for null so tiles never display NaN.	Module-level formatter reuse for performance
<code>utils/risk.js</code>	87	Presentation mirror of the engine's bands, alert type labels, status styles, and latency colour coding against the 50 ms budget.	Single source of visual truth for risk semantics

File	Lines	Responsibility	React concepts
<code>data/modelFacts.js</code>	99	Real training figures hard-coded so the public pages can quote metrics without an authenticated API call, with the refresh procedure documented in the file.	Deliberate constant duplication with a documented provenance chain

14.2 Component modules

File	Lines	Contents
<code>components/ui.jsx</code>	262	Fourteen primitives: Card, CardHeader, Badge, RiskBadge, StatTile, Bar meter, hand-built SVG Sparkline, EmptyState, Spinner, Banner with a status role, Tabs with tab semantics, Skeleton, TableShell for horizontal scroll, and DefinitionRow. Percentages are clamped so bad data cannot overflow a bar.
<code>components/Icons.jsx</code>	103	Forty inline stroke icons plus the logo and wordmark. Hand-rolled to avoid an icon font dependency; unknown names return null rather than crashing; every icon is hidden from assistive technology because its button carries the label.
<code>components/app/AppShell.jsx</code>	214	Sidebar with seven navigation entries and two live badges (open alerts, active cases), off-canvas below large screens, a sticky top bar with breadcrumb, the realtime status pill with five states, the live region, and the page header primitive.
<code>components/app/StreamControls.jsx</code>	237	The only write control in the shell: start and stop, plus a settings dialog with transaction limit, three arrival-rate presets, three start positions derived from the fraud index, and a persistence toggle. Restarting stops first because the engine refuses a second concurrent stream.
<code>components/app/tables.jsx</code>	400	Four surfaces: the transaction feed with keyboard-activatable rows, the transaction detail drawer with behavioural context and a ground-truth panel, the alert queue with inline status selects and deep links into the workbench, and the account table with expandable signal bars.
<code>components/app/widgets.jsx</code>	469	The only module importing the chart library. Four charts plus five non-chart panels: risk distribution ribbon, latency panel with per-percentile meters, confusion matrix grid, account signal bars, candidate comparison table and the risk band legend. All charts share one axis style and one custom tooltip.
<code>components/landing/*</code>	792	Three files: the site chrome with a scroll-aware sticky header and mobile navigation, seven content sections built from the real model facts, and a hero preview that is explicitly captioned as a sample rather than live data and that respects reduced-motion preferences.

One honesty detail in the interface itself

The landing page shows a dashboard preview with moving rows. Because it is not connected to the engine, it carries a visible caption stating that the rows are a fixed sample from the held-out test split. Marketing surfaces are where projects usually overstate, so we labelled it instead.

15 Advanced feature: the explainable investigation workbench

This is the newest and most substantial addition, and the one that turns a detector into a tool an analyst can actually work with. It spans all four layers: model explainability, backend state, database schema and a full user interface.

15.1 The problem it solves

Before this feature the system could tell an analyst that transaction TXN-0001326 scored 0.999993 and was critical. It could not tell them *why*, could not record what the analyst decided, and could not measure whether the model's alerts were actually correct. Three consequences followed: no auditability, no coordination between analysts, and no feedback signal for the next training round.

15.2 Layer one: explainability with native TreeSHAP

Aspect	Implementation
Method	XGBoost's built-in contribution prediction, which returns exact additive TreeSHAP values plus a bias term.
Output space	Raw margin, that is log-odds. Contributions sum with the base value to reproduce the margin, and the sigmoid of that margin reproduces the probability. The report states this explicitly so nobody misreads a contribution as a probability delta.
Verification built in	The payload carries both the model probability and the probability reconstructed from the contributions. On the verified live case both read 0.999982, which proves the explanation corresponds to the actual decision.
Cost control	Explanations are computed only for alerting events, and deliberately after the latency measurement, so the 50 ms budget is untouched. Non-alerting traffic pays nothing.
Ranking	Features are sorted by absolute contribution and truncated to the top eight, each labelled as raising or reducing risk, with the observed feature value shown alongside.
Graceful degradation	Any non-tree estimator, or any failure inside the call, returns an explicit unavailable result with a reason string. The case still opens with reason codes, and the interface shows an "Unavailable" badge instead of an empty panel.

Alongside the mathematical explanation, every case carries the behavioural reason codes from section 9.5. The two answer different questions: TreeSHAP answers "which model inputs moved this score", reason codes answer "what about this behaviour is unusual". An analyst needs both.

15.3 Layer two: case lifecycle with optimistic concurrency

Capability	Behaviour
Automatic creation	Every alert opens a case with a readable number, a priority derived from the risk band, an immutable transaction and alert snapshot, the explanation, the reason codes and a seeded creation event.
Duplicate suppression	If a still-open case exists for the same transaction, it is reused rather than duplicated. This matters because restarting a stream without a reset replays rows that were already scored.
Assignment	Ownership can be claimed or released, recording who assigned whom and when.
Notes	Append-only, length-limited, attributed, and never editable, so the trail cannot be quietly rewritten.

Capability	Behaviour
Status transitions	Open, investigating, resolved or dismissed, mirrored onto the underlying alert so the triage queue and the workbench cannot disagree.
Resolution	Six outcome codes, a written rationale and an optional analyst confidence value. A duplicate resolution dismisses the case rather than resolving it.
Audit trail	Every mutation appends an event with actor, timestamp and a human-readable description.
Analyst bias guard	The dataset's ground-truth label is stripped from the case snapshot. An analyst must judge the evidence, not read the answer.
Bounded memory	Cases beyond a 2,000-case ceiling are evicted oldest-first, and both lookup indexes are cleaned with them.

Why optimistic concurrency rather than locking

Two analysts can open the same case. If both submitted a status change, a last-write-wins design would silently discard one decision. Instead every case carries a version that increments on every mutation, and every write includes the version the analyst was looking at. A mismatch returns **409 Conflict** with a message telling them to refresh. This is the same technique databases use for row versioning, applied across HTTP requests, and it holds no lock while a human thinks.

15.4 Layer three: API and persistence

Seven endpoints were added, all additive, so no existing response shape or latency characteristic changed: list cases, workflow metrics, case detail, assignment, notes, status and resolution. Two tables were added in a separate migration, both read-only for the browser, with write access reserved for the engine's service role. Cases are upserted on their identifier so a repeated flush is harmless, and notes are inserted append-only.

15.5 Layer four: the analyst interface

Element	What it does
Four workflow tiles	Active cases, unassigned count, confirmed fraud outcomes, and mean resolution time with the analyst confirmation rate.
Filterable queue	Status tabs including an "active" grouping, plus a risk band filter. Each row shows the primary evidence, whether an explanation is available and the note count.
URL-addressable drawer	An open case is encoded in the query string, so a case can be linked in a chat message or bookmarked. Closing replaces history rather than pushing, so the back button behaves sensibly.
Evidence panel	Signed contribution bars normalised against the largest absolute contribution, coloured by direction, with the observed value and a plain-language explanation of the log-odds space.
Workflow controls	Assign to me, status select, note composer, and a resolution form with outcome, confidence and rationale. Every submission carries the expected version.
Audit view	Two parallel timelines, newest first: workflow events and analyst notes, both attributed and relatively timestamped.
Deep links from alerts	Any alert with a case becomes a link straight into that case, so triage and investigation are one motion rather than two screens.
Live collaboration	Because case and note changes are published over Realtime, another analyst's edit appears without a manual refresh.

15.6 Verified end to end

Check	Observed result
Case opened from a live alert	Case number generated, one reason code <code>critical_model_risk</code> , risk score 0.999993
Explanation present and faithful	Eight ranked contributions; model probability and reconstructed probability both 0.999982
Version chain across four mutations	1 to 2 on assignment, 3 on status, 4 on note, 5 on resolution
Alert synchronisation	Underlying alert status followed the case to resolved
Workflow metrics	Confirmed fraud 1, confirmation rate 1.0, mean resolution recorded in seconds
Duplicate suppression	Replaying the same rows raised a second alert but reused the same case, count stayed at one
Browser rendering	Drawer, evidence, reason codes, notes and resolution all rendered with zero console errors, zero page errors and zero failed requests

Why this feature is the right one to demonstrate

It converts three abstract claims into visible artifacts. Explainability stops being a buzzword when a panel can see eight signed contributions that reconstruct the exact probability. Auditability stops being a promise when every action has an actor and a timestamp. And the resolution codes create the feedback loop that a production fraud programme runs on: the confirmation rate is the first honest measurement of whether the model's alerts are worth an analyst's time.

16 Security model

16.1 Key custody

Credential	Lives in	Capability and reasoning
Service role key	Server environment file only	Bypasses row level security entirely. It is never referenced in any frontend file, and the frontend README states the rule explicitly. The build tool inlines only variables with the public prefix, so it cannot leak into the bundle by accident.
Anon key	Browser bundle and server	Safe to publish because row level security governs what it can reach. The server uses it only to verify user tokens.
User access token	Browser session storage, managed by the auth library	Attached as a bearer token to every API call and verified server-side. Short lived and auto-refreshed.

16.2 Defence layers

Layer	Control
Transport surface	CORS restricted to four explicit development origins, methods limited to GET, POST, PATCH and OPTIONS, and only three request headers allowed. No PUT or DELETE exists anywhere.
Identity	Every route except health requires a bearer token, verified against the auth provider with a 60-second bounded cache.
Input validation	Declarative schemas enforce numeric ranges, regex-constrained statuses, string length caps and unknown-field rejection before any handler runs.
File system	Stream sources must be a bare CSV file name; six traversal payloads are test-asserted to fail. Uploads are extension-checked, character-allow-listed, streamed in chunks, capped at 250 MB mid-flight, and partial files are deleted on breach.
Database rows	Row level security on all seven tables. The anonymous role reaches nothing. A user sees only their own profile row.
Database columns	Analysts may update the alert status column and nothing else, enforced by grant rather than application logic.
Write path	Investigation tables are read-only for the browser. All case mutations pass through the API, which records the actor and applies the version check.
Data minimisation	No real cardholder data exists in the project. Account, card suffix, merchant and location are deterministic hashes of already-anonymised components.

16.3 Known weaknesses, stated plainly

Weakness	Impact	Recommended fix
Authentication fails open	If the require-auth flag is set but either Supabase value is missing, every protected route accepts requests as a development user with only a log warning. A misconfigured deployment would be publicly writable.	Refuse to start when authentication is required but unverifiable, so the failure is loud at boot rather than silent at runtime.

Weakness	Impact	Recommended fix
No role authorisation	The profile role column exists with four values, but no endpoint checks it. Any authenticated user can start a stream, upload a dataset or resolve a case.	Add a role requirement to the auth dependency and restrict stream control and upload to admin or developer roles.
Users can edit their own role	The profile update policy covers the whole row, so a user could set their own role to admin. Harmless today because roles are unenforced, but it becomes an escalation path the moment they are enforced.	Revoke update on the role column, mirroring the alert status pattern already used.
Demo mode bypasses sign-in	A build flag allows entering the console with no credentials.	Defaults to off and is documented as local-only. Keep it out of deployed builds; the code comment says so and the login screen warns on screen.
In-memory state is not durable	Restarting the engine clears rolling buffers and open cases. Rows already written to the database survive.	Rehydrate cases from the database at startup; noted in the roadmap.

Why we list these rather than omit them

Each item is a deliberate development-time convenience with a known production fix, not an oversight we discovered while writing this report. A reviewer who finds an undisclosed fail-open auth path will discount everything else in the document; a reviewer who sees it disclosed with a one-line remedy reads it as engineering maturity.

17 Performance engineering

17.1 Where the milliseconds go

MEASURED BREAKDOWN OF THE TIMED INFERENCE PATH

Stage	Approximate cost	What happens
Build the feature vector	0.05 ms	Fill a pre-allocated 33-slot array; no DataFrame allocation
Impute and scale	0.15 ms	Two vectorised operations from the serialised pipeline
Model forward pass	0.75 ms	400 shallow trees, histogram method
Total measured average	0.9739 ms	p95 1.588 ms, p99 1.932 ms, maximum observed 2.291 ms
Budget	50 ms	Roughly 98 % of the budget remains unused

17.2 The seven decisions that produced that number

Decision	Effect
Latency measured through the real serving path	The training benchmark calls the same three functions the API calls, so the published number is not a laboratory figure that disappears in production.
Warm-up before measuring	Twenty untimed records absorb lazy allocation, and the predictor also warms itself at startup, so the first real request is not an outlier.
Latency gate in model selection	Rejected a model that was 0.024 PR-AUC better but 19 times slower.
Scalar feature path	Avoids constructing a DataFrame per transaction, which would dominate a sub-millisecond budget.
Persistence off the hot path	Database writes are queued for a background thread, so network latency and retry behaviour never count against inference.
Explanations only for alerts, after timing	TreeSHAP runs on roughly 0.1 % of traffic and never inside the measured section.
Metrics served from memory	The dashboard endpoint reads ring buffers rather than running aggregate queries, so a 2-second poll cannot degrade scoring.

17.3 Memory behaviour

Memory is bounded by construction, not by hope. Every buffer has a hard maximum, so a stream can run indefinitely without growth: 5,000 latency samples, 500 transactions, 500 alerts, 120 timeline buckets, 64 recent events and locations per account, 2,000 investigation cases and a 10,000-row write queue. The generator holds one row at a time, and the training pipeline explicitly releases intermediate frames between stages.

What we measured versus what we did not

Latency and throughput are measured on a single development machine with the stream running as fast as the file can be read, reaching roughly 312 transactions per second in a verified run with zero inter-arrival delay. We have **not** load-tested concurrent API clients, and we have not measured behaviour under a real broker. Section 22 lists both as open work rather than implying they are done.

18 Testing and verification

18.1 The automated suite: 60 passing cases

Test file	Cases	What it proves
<code>test_api.py</code>	17	Health is public and reports readiness; every specified endpoint exists in the OpenAPI document; prediction returns a scored transaction inside the latency target; malformed bodies are rejected; the full stream lifecycle produces coherent metrics; six path-traversal payloads are refused; alert triage rejects unknown identifiers and invalid statuses.
<code>test_risk.py</code>	17	The tuned threshold maps to exactly the high band edge for four different thresholds; endpoints and monotonicity hold across a 101-step sweep; band boundaries match the specification; account risk aggregates repeated suspicion; behavioural features use prior activity only; the seven weights sum to one.
<code>test_generator.py</code>	10	The stream really is a generator; events arrive one at a time; no batch read occurs , proven by counting parse calls; every event carries an identifier, timestamp and identity, and never the label; a malformed row is skipped and counted; stopping mid-file loses nothing; limit and skip behave.
<code>test_processor.py</code>	7	Exactly one prediction per event; every dashboard field is present; high amounts raise alerts and escalate the account; labels feed the live confusion matrix and its cells sum to the processed count; the metrics payload has the right shape; a real background thread starts, stops and restarts; counters reset between runs.
<code>test_features.py</code>	6	The scalar and vectorised feature paths are numerically identical; the vector has the right shape and column order; malformed input still yields finite values; night boundaries are exact; derived identity is deterministic and none of its keys are model features.
<code>test_latency.py</code>	3	Across 300 samples with 25 discarded warm-ups, both the average and the p95 stay inside the 50 ms target; every prediction returns a valid probability; the recorded metadata agrees with the live measurement.
Total	60	Verified passing in 2.9 seconds, zero failures, zero errors, zero skips on this machine.

These tests assert architecture, not just output

Three of them are unusual and worth pointing at in a review. The parse-counting test proves the stream is lazy, which is an architectural claim. The feature parity test proves two code paths cannot drift. The weight normalisation test proves the risk model stays a weighted average even if someone edits a constant. Those are properties, not values, and they are the ones that break silently without a test.

18.2 Beyond unit tests

Harness	What it does
End-to-end smoke script	Spawns a real server on its own port and asserts 23 checks over HTTP, including that a second concurrent stream start returns 409, that risk distribution counts sum to the transaction total, that account signal breakdowns contain all seven values, and that an uploaded CSV can be streamed and then cleaned up.

Harness	What it does
Interaction harness	Drives the real interface in a browser: landing anchors, the auth guard redirect and return-to-page, the stream settings dialog, KPI tiles counting up, monitor filters and search, pause and resume, the detail drawer, alert triage cross-checked against the API, account expansion, the 404 page, sign-out restoring the guard, and a mobile viewport pass.
Console screenshot harness	Starts a stream near labelled fraud, waits for completion, signs in and captures every console page, reporting heading text, row counts and any console error or 404 per page.
Continuous integration	Two jobs on every push: the Python suite on 3.12, and a production frontend build on Node 20. Tests that need trained artifacts skip automatically, so CI does not require a 150 MB dataset.

18.3 Verification performed for this report

Check	Result
Python test suite	60 passed 0 failed, 0 errors, 2.9 s
Production frontend build	Success 992 modules transformed in 4.71 s
Static diagnostics across changed files	0 errors
Live stream run	Pass 40 transactions, 1 alert, 1 case, 0 invalid, ~312 per second
Investigation lifecycle over HTTP	Pass version chain 1 to 5, alert synchronised
Browser verification	Pass 0 console errors, 0 page errors, 0 failed requests, 0 responses at or above 400

19 Engineering rigour: a bug we found and fixed

While verifying the workbench we found a real defect. It is included here deliberately, because how a team finds and fixes a bug says more than a list of features.

Symptom

The investigations page listed seven active cases while the alerts page was empty. Cases were being shown for fraud that no longer existed in the system.

Diagnosis

Live inspection confirmed the divergence exactly: **0 alerts in the buffer, 7 cases**, and all seven case transaction identifiers had no matching alert.

19.1 Root cause

Starting a stream with the reset option clears three things: the transaction feed, the alert buffer and account risk. When the investigation store was added, it was never included in that reset. So cases outlived the run that created them, and their alert links dangled. A second defect shared the same origin: replaying the same rows without a reset opened a duplicate case for a transaction that already had one.

19.2 The fix

Change	Detail
Reset method on the case store	Clears cases and both lookup indexes under its lock, with a comment explaining that database rows are the durable history and are deliberately left alone.
Wired into the stream lifecycle	A resetting stream now clears cases alongside the alerts they belong to.
Duplicate guard	If a case for the same transaction is still open or investigating, it is reused. Closed cases still allow a genuinely new investigation.

19.3 Verification of the fix

Scenario	Expected	Observed
Baseline after restart	0 alerts, 0 cases	Match
Fraud run, 40 rows at the labelled-fraud offset	1 alert, 1 case, identifiers matching	Match
The reported scenario: 18-row reset run with no fraud	0 alerts, 0 cases, all metrics zero	Match
Replay without reset	Second alert raised, case count stays 1, same case identifier	Match
Interface after the fix	Empty state, all tiles zero, no sidebar badge	Match, 0 console errors
Regression check	No existing behaviour broken	60/60 tests still pass

The transferable lesson

The bug was not in the new feature's logic; the case store worked correctly in isolation. It was in the **lifecycle contract** between an existing subsystem and a new one. When a system has several stores that must be cleared together, "reset" is a cross-cutting concern, and adding a store means joining that contract. That is exactly the class of defect that unit tests on the new component alone will never catch, and it is why we verified the feature against a running system rather than only against its own tests.

20 Running the project

20.1 One-time setup

```
# 1. Python environment for the detection engine
cd ml-engine
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt

# 2. Frontend dependencies
cd ..\frontend
pnpm install          # or npm install

# 3. Environment files, copied from the templates and filled in
#   ml-engine\.env      : SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, SUPABASE_ANON_KEY
#   frontend\.env.local : VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY, VITE_API_URL

# 4. Database schema: run both migrations in the Supabase SQL editor
#   supabase\migrations\0001_init.sql
#   supabase\migrations\0002_investigations.sql
```

20.2 Train the model

```
# Full training run, roughly 75 seconds, writes all six artifacts
cd ml-engine
.\.venv\Scripts\python.exe -m src.training.train --latency-samples 500

# Fast pass for a quick check
.\.venv\Scripts\python.exe -m src.training.train --max-rows 30000 --fast

# Useful flags
#   --models xgboost,random_forest  train a subset
#   --smote                          enable optional oversampling
#   --no-stream-file                 skip writing the replay file
```

20.3 Run both services

```
# Terminal 1: detection engine on port 8000
cd ml-engine
.\.venv\Scripts\python.exe -m uvicorn src.api.main:app --host 127.0.0.1 --port 8000

# Terminal 2: dashboard on port 5173
cd frontend
pnpm dev

# Then open http://127.0.0.1:5173 and sign in.
# Start the stream from the top bar; choose "At the first labelled fraud"
# so alerts and investigation cases appear within seconds.
```

20.4 Verification commands

Command	Purpose
<code>python -m pytest -q</code>	The 60-case suite
<code>python scripts\smoke_e2e.py --limit 400</code>	End-to-end HTTP checks against a spawned server
<code>tools\verify-all.ps1</code>	Tests plus smoke run on an isolated port
<code>tools\ui-flows.ps1</code>	Browser interaction suite
<code>tools\verify-console.ps1</code>	Build, preview and screenshot every console page
<code>pnpm build</code>	Production bundle

Two operational constraints

Run the engine with a **single worker**, because the stream is an in-process background thread and rolling state is per-process. And point the dashboard at the engine with the API URL variable; if the engine is unreachable the console says so explicitly, naming the URL it tried.

21 Concept glossary

Every technical term this report uses, in plain language, for team members and reviewers who are not the authors.

21.1 Machine learning

Term	Meaning in this project
Class imbalance	One outcome is far rarer than the other. Here fraud is 1 in 599 transactions, which breaks naive metrics and default thresholds.
PCA component	Principal Component Analysis output. The publishers replaced the original fields with 28 mathematical combinations to anonymise them. They carry signal but are not human-readable.
Precision	Of the transactions we flagged, the share that really was fraud. Low precision means analysts waste time.
Recall	Of all real fraud, the share we caught. Low recall means losses.
F1 score	Harmonic mean of precision and recall, so it punishes a model that is good at one and bad at the other. Our test F1 is 0.75.
F2 score	Same idea, but recall counts four times more than precision. We tune the threshold on F2 because missing fraud costs more than reviewing a false alarm.
Accuracy	Share of all predictions that were correct. Ours is 99.94 %, but a do-nothing model scores 99.88 %, which is why we never rely on it.
Balanced accuracy	Average of recall and specificity, so each class counts equally regardless of size. Ours is 87.48 % on test, a far more informative figure.
PR-AUC	Area under the precision-recall curve. Summarises quality across all thresholds while ignoring the huge easy negative class. Our headline metric at 0.7629.
ROC-AUC	Area under the receiver operating characteristic curve. Measures ranking quality, but looks flattering under heavy imbalance.
Matthews correlation	A single correlation-style score using all four confusion matrix cells. Robust to imbalance; ours is 0.7497 on test.
Confusion matrix	The four outcomes: true positive, false positive, false negative, true negative. Everything else is derived from these.
Threshold	The probability above which we act. Not a model property but a business decision. Ours is 0.184152.
Data leakage	When information unavailable at prediction time influences training, producing scores that collapse in production. Prevented by the chronological split, fitting the scaler on training data only, and an explicit column check.
Chronological split	Dividing data by time rather than at random, so the model is always tested on the future relative to its training data.
Standardisation	Rescaling each feature to zero mean and unit variance so features on different scales contribute comparably.
Class weighting	Telling the model that mistakes on the rare class cost more, rather than duplicating rare rows. Our primary imbalance strategy.
SMOTE	Synthesising new minority examples by interpolating between real ones. Available behind a flag, off by default.

Term	Meaning in this project
Gradient boosting	Building many small decision trees in sequence, each correcting the previous ones' errors. The technique behind the selected model.
TreeSHAP	An exact method for splitting a tree model's prediction into per-feature contributions that sum back to the prediction. The basis of our explanations.
Log-odds	The additive space tree models work in. A sigmoid converts it to a probability. Our contributions are expressed in this space, which the interface states on screen.

21.2 Software and systems

Term	Meaning in this project
Generator	A Python function that yields one item at a time and remembers where it stopped. Gives us streaming semantics with constant memory.
Ring buffer	A fixed-capacity queue that overwrites its oldest entry when full. Keeps dashboard memory constant no matter how long a stream runs.
Thread	An independent line of execution inside one process. We use three kinds: request handlers, the scoring loop, and the database writer.
Lock and re-entrant lock	Mechanisms ensuring one thread at a time touches shared state. Re-entrant means the same thread may acquire it again, which we need because internal helpers call each other.
Producer and consumer queue	One thread enqueues work, another drains it. Decouples scoring speed from database speed.
Back-pressure and load shedding	When a queue fills, you either slow the producer or drop work. We drop and count, because pausing fraud scoring to wait for a database write would be the wrong trade.
Optimistic concurrency	Instead of locking a record while a human decides, you attach a version number and reject a write that was based on stale data. Produces our 409 responses.
Idempotent	Safe to run more than once with the same result. Our migrations and our case upserts are idempotent.
Singleton with double-checked locking	Creating an expensive object exactly once per process without paying lock cost on every subsequent access. Used for the model, the processor and the writer.
Welford's algorithm	Computing a running mean and variance in one pass with constant memory and better numerical stability than summing squares.
Percentile, p95 and p99	The value below which that share of samples falls. We report p95 and p99 latency rather than only the average, because tail behaviour is what users feel.
Row level security	Database rules deciding which rows a role may see or change, enforced by the database itself rather than by application code.
Column-level grant	Permission to modify specific columns only. How we let analysts change an alert's status but never its score.
JWT bearer token	A signed, short-lived credential sent with each request in the authorisation header, verified server-side.
Path traversal	An attack that uses <code>..</code> or absolute paths to reach files outside the intended directory. Blocked by a validator and asserted by six tests.
CORS	Browser rules governing which origins may call an API. Ours allows four explicit development origins and four methods.

Term	Meaning in this project
Realtime replication	The database streaming its changes to subscribers, so the dashboard updates on write instead of waiting for the next poll.
Code splitting and lazy loading	Deferring download of code until it is needed. The console chunk is not fetched by visitors to the landing page.
Optimistic UI update	Showing the expected result immediately and reconciling with the server afterwards, so the interface feels instant.
Context provider	A React mechanism for sharing state with a whole subtree without passing props through every level. One provider holds all live data.
Fail open and fail closed	What a system does when a security check cannot run. Ours currently fails open for authentication, which section 16 flags with the fix.

22 Limitations and roadmap

22.1 Honest limitations

Limitation	Why it exists	What it means for the results
Two days of 2013 data	The public dataset covers a 48-hour window from one European card issuer.	Nothing here proves seasonal or regional generalisation. Fraud patterns have also moved on since 2013.
Only 52 frauds in the test split	Chronological splitting of a rare event leaves few positives in the final window.	Confidence intervals are wide. Three transactions crossing the boundary move recall by about six points, which is why validation and test F1 differ by 0.10.
Anonymised features	The publishers replaced original fields with PCA components.	An explanation can say V14 pushed the score up, but not what V14 represents in business terms. With real fields the same TreeSHAP output would name a merchant category or a device.
Threshold is dataset-specific	0.184152 was tuned on this validation split.	It must be re-tuned for any other data source or cost structure. It is stored in metadata, not hard-coded, precisely so it can move.
Synthetic account attributes	The dataset has no account, merchant or location fields.	Account-level aggregation demonstrates the mechanism correctly, but the groupings are hash-derived, not real customer relationships.
No drift monitoring	Out of scope for this iteration.	Nothing detects the model degrading over time. The live confusion matrix is the closest available signal.
Single-process state	Rolling buffers and cases live in memory.	The engine must run one worker, and a restart clears open cases although persisted rows survive.
Not load tested	We measured single-transaction latency and single-client throughput only.	Behaviour under many concurrent dashboard clients is unmeasured, and we do not claim it.
Documentation drift	One README still cites an older test count of 53 while the suite now has 60.	Cosmetic, and disclosed here rather than left for a reviewer to find.

22.2 Roadmap, in priority order

Priority	Increment	Rationale and approach
1	Fail closed on authentication, and enforce roles	Refuse startup when auth is required but unverifiable; add a role requirement to the dependency; revoke update on the profile role column. Small change, removes the largest disclosed risk.
2	Durable case state	Rehydrate investigation cases from the database at startup so an engine restart does not lose analyst work in progress.

Priority	Increment	Rationale and approach
3	Feedback-driven retraining	Analyst resolution codes are already captured. Feed confirmed fraud and confirmed false positives back as labels, and track the confirmation rate as a model health metric between training runs.
4	Drift monitoring	Track feature distributions and the live precision estimate over time, and alert when either moves beyond a tolerance.
5	Real broker ingestion	Replace the generator with a Kafka or Redis Streams consumer. Everything downstream already assumes single-event processing, so this is a source swap plus consumer-group state.
6	Multi-worker architecture	Move rolling state into Redis so several workers can serve the dashboard consistently and the stream can run as a separate process.
7	Accessibility completion	Focus trap and Escape handling in the transaction drawer, roving focus in the tab groups, and text alternatives for charts.
8	Cost-weighted thresholds	Replace the fixed precision floor with an explicit cost matrix so the threshold optimises expected monetary loss rather than a statistical proxy.

23 Panel question preparation

The questions most likely to be asked, with answers grounded in evidence from this project.

What is your F1 score and your accuracy?

On the held-out test split, **F1 is 0.75** with precision and recall both exactly 0.75, and **accuracy is 99.94 %**. On validation, F1 is 0.8491 and accuracy is 99.96 %. We lead with F1 because accuracy is nearly uninformative here: predicting "legitimate" for every transaction scores 99.88 % on the same split while catching zero fraud. Our balanced accuracy is 87.48 % and our Matthews correlation is 0.7497, which are the figures that actually separate our model from that constant function.

Why is your accuracy so high but your F1 comparatively modest?

Because 42,508 of 42,560 test transactions are legitimate and we classify almost all of them correctly, which dominates accuracy. F1 looks only at the 52 frauds and the 52 alerts: we caught 39, missed 13 and raised 13 false alarms. That is the number that reflects the actual job.

Why did you not choose the model with the best score?

Random Forest scored PR-AUC 0.8715 against XGBoost's 0.8472, so on quality alone it wins. Its p95 latency was 33.16 ms out of a 50 ms budget, on an idle laptop, leaving nothing for a loaded production host. We apply a hard gate at 50 ms and a comfort filter at 25 ms, then take the best PR-AUC among survivors. We traded 0.024 PR-AUC for a 19-fold latency improvement, and the training run logs that decision explicitly.

How do you know there is no data leakage?

Four independent controls. The split is chronological, so training data always precedes test data. Duplicates are removed before splitting, so a row cannot appear on both sides. The imputer and scaler are fitted on the training split only. And an explicit check refuses any feature whose name resembles a label, recorded as passed in the model metadata. The derived account and merchant attributes are also excluded from the feature list, with a test asserting it.

Is this real streaming, or are you replaying a file?

It is a replay, and we call it pseudo-streaming rather than dressing it up. What matters architecturally is that processing is one record at a time with bounded memory and a clean stop, which a Python generator gives us and a test proves by counting parse calls. Replacing the generator with a broker consumer changes the source of the loop, not the loop, because everything downstream already handles single events.

How would this scale to production volumes?

Inference has roughly 98 % of its budget unused, so per-transaction cost is not the constraint. Three things would change: swap the generator for a broker consumer with consumer groups, move rolling state into Redis so multiple workers stay consistent, and run the scoring loop as its own process. The persistence layer already batches and sheds load, so it is the part least likely to need redesign. We have not load tested concurrent clients, and we do not claim a throughput figure we have not measured.

What happens when the model is wrong?

Both directions are handled. A false positive becomes a case an analyst can resolve as a model false positive, which is recorded as feedback. A false negative shows up in the live confusion matrix whenever the replayed data carries a label. Neither is silent. The resolution codes are the beginning of the retraining loop described in the roadmap.

Can you explain a specific decision to a customer or a regulator?

Yes, at two levels. TreeSHAP gives exact additive contributions per feature, and we verify fidelity by reconstructing the probability from those contributions: on a verified live case both the model probability and the reconstruction read 0.999982. Reason codes then translate the behaviour into words with the observed value and the threshold crossed. The honest limitation is that the dataset's features are anonymised components, so we can say V14 drove the score but not what V14 means in business terms.

What is the weakest part of this project?

Authentication currently fails open: if the auth flag is set but a Supabase key is missing, every protected route accepts requests as a development user with a single log warning. It is convenient locally and dangerous in a deployment. The fix is to refuse startup in that state, and it is the top item on our roadmap. The second weakest point is statistical: 52 test frauds is a small sample, so our metrics carry wide confidence intervals.

What did you learn that you did not expect?

That the most valuable bug was in a contract, not in an algorithm. Our case store passed its own tests, but it had not joined the lifecycle contract that clears state on a stream reset, so the workbench showed cases for alerts that no longer existed. Component-level tests could not catch it. That is why we verify features against a running system, and it is documented in section 19 rather than quietly fixed.

How much of this is your own work rather than a library?

The libraries provide the estimators, the HTTP framework and the chart primitives. What we built is roughly **14,100 lines across 97 source files**: the risk mapping and account aggregation mathematics, the streaming loop and its concurrency model, the latency-gated selection rule, the explanation and case workflow layers, the database security model, the whole interface, and the verification harnesses.

24 Code inventory

24.1 Source code by area

Area	Files	Lines	Contents
Python engine	37	6,506	Config, preprocessing, features, training, inference, risk, streaming, investigations, persistence, API, tests, smoke script
React frontend	38	6,167	Entry points, 10 pages, 12 component modules, 2 contexts, 2 hooks, services, utilities, styling and build configuration
Database SQL	3	540	Two migrations and a demo seed file
Automation tooling	19	890	15 PowerShell scripts and 4 Playwright harnesses
Total source	97	14,103	Excluding documentation, generated artifacts and dependencies

24.2 Largest modules

File	Lines	Role
ml-engine/src/training/train.py	709	Pipeline orchestrator and CLI
ml-engine/src/api/main.py	643	All 21 endpoints and lifecycle hooks
frontend/src/pages/Investigations.jsx	579	Case workbench and evidence drawer
ml-engine/src/risk/scoring.py	489	Risk mapping and account aggregation
ml-engine/src/streaming/processor.py	489	Stream lifecycle and per-event pipeline
frontend/src/components/app/widgets.jsx	469	All charts and analytics panels
ml-engine/src/investigations/store.py	445	Case state with optimistic concurrency
frontend/src/components/landing/Sections.jsx	432	Public marketing sections
frontend/src/components/app/tables.jsx	400	Four data tables and the detail drawer
frontend/src/context/StreamContext.jsx	397	Live data store, polling and Realtime

24.3 Generated artifacts, not source

The following are produced by running the project and are excluded from the counts above: the two serialised model files and the metadata JSON, the replay CSV, the dataset profile, the fraud index, the frontend production bundle, dependency directories, log files, screenshots and the rendered versions of this report. The raw dataset itself lives outside the repository because of its size, and the engine searches three candidate locations to find it.

Closing statement

FraudStream AI began as a requirements document and a CSV file. It now trains and compares four models on 283,726 real transactions, ships the one that satisfies both a quality and a latency requirement, scores unseen transactions in under a millisecond, escalates accounts on seven weighted behavioural signals, explains individual decisions with verifiable additive contributions, and gives an analyst a complete case workflow with an audit trail.

Test F1 is 0.75, PR-AUC 0.7629, false positive rate 0.03 %, p95 latency 1.588 ms against a 50 ms budget, and 60 of 60 automated tests pass. Where the project falls short, sections 16 and 22 say so and name the fix. That combination, measured results with disclosed boundaries, is what we would want to be judged on.

FraudStream AI · Complete technical report · NPN Team 47 · 13 August 2026. Model figures from `ml-engine/models/model_metadata.json` (XGBoost v1.0.0, trained 13 August 2026). Dataset: ULB credit card fraud dataset. No production cardholder data was used in this project.